دانشگاه تهران

پردیس دانشکده‌های فنی

دانشکده‌ی مهندسی برق و کامپیوتر

تمرین دوم درس مبانی علوم شناختی

نام و نام خانوادگی:

سینا پیرمرادیان

شماره دانشجویی:

۸۱۰۱۰۱۱۲۵

استاد:

دکتر محمدرضا ابوالقاسمی دهاقانی

Introduction:

The first part of the exercise is related to the Spike Sorting analysis. With the ability to record electrical signals from individual neurons, researchers have developed powerful tools and techniques for studying brain function. One such technique is spike sorting, which involves a sequence of mathematical techniques to accurately infer the spiking activities of every neuron that is sufficiently close to the electrode. In this question, we will perform spike-sorting analysis on a single-channel recording of a population of simulated neurons, where we will be guided through every step of the analysis.

The steps include filtering, spike detection, feature extraction, and clustering. Several clustering algorithms are available for spike sorting.

1. Spike Sorting from Scratch

In this section, we intend to reveal spikes related to Action Potential using the sample recorded by Cell single and find the number of neurons through clustering.
I write First part, both in Matlab and python which was attached in my zip file.

Getting Started:

a) In this part, at first, we load the data into the data variable using the scipy library as shown in Figure 1, we load the data recorded by the single cell, in order to get the time axis, we use the given sampling frequency, i.e. 2400 Hz. The code related to this part is shown in Figure 2 and its output is shown in Figure 3.

b)

Loading Data

```
[ ]  1 # Load the data from the .mat file
     2 data = sio.loadmat('/content/drive/MyDrive/Cognetive/HW_2/extracellular.mat')
     3
     4 spike = sio.loadmat('/content/drive/MyDrive/Cognetive/HW_2/spikes.mat')
     5 spike_idx = spike['SpikeInds'][0]
```

Figure 1

Extracextract the Voltage Amplitude

```
 1 # extract the voltage amplitude data from the structure
 2 voltage = data['all_data_with_noise_and_line'][0,:]
 3
 4 # Sampling rate of the recording is 2400Hz
 5 sampling_rate = 2400
 6
 7 # Create a time axis based on the sampling rate of 2400Hz
 8 sampling_rate = 2400
 9 time = np.arange(len(voltage)) / sampling_rate
10
11 # Plot the voltage against time
12 plt.plot(time, voltage)
13 plt.xlabel('Time (s)')
14 plt.ylabel('Voltage (mV)')
15 plt.title('Voltage vs. Time')
16 plt.show()
```

Figure 2

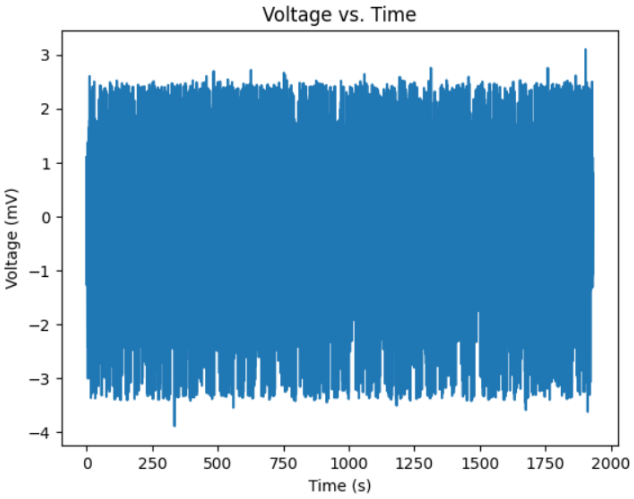

Figure 3

Moreover, we plot the voltage against the time in a wide variety, in Figure 4 we write the code and in Figure 5 the input waveform are ploted.

Plot the voltage and against time in a wide shape

```
[6]  1 # Plot the voltage against time
     2 plt.figure(figsize=(40,2))
     3 plt.plot(voltage)
     4 plt.title('amplitude of Dataset')
     5 plt.xlabel('time')
     6 plt.ylabel('amplitude')
```

Figure 4

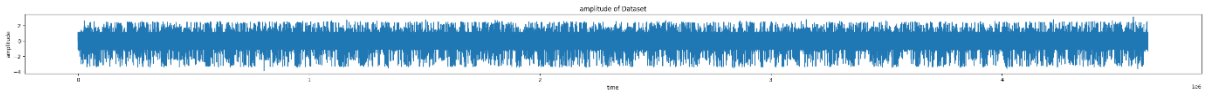

Figure 5

b) In this part, we are going to plot the histogram related to the voltage range recorded by single cell, which is done in Figure 6,7. As it appears from the figure, we have a Gaussian mixture shape, which shows that the noise is Gaussian.

Plot the histogram of the voltage amplitudes

```
[7]  1 # Flatten the voltage array to a 1D array
     2 voltage_flat = voltage.flatten()
     3
     4 # Plot the histogram of the voltage amplitudes
     5 plt.hist(voltage_flat, bins=100, edgecolor='black')
     6 plt.xlabel('Voltage (mV)')
     7 plt.ylabel('Count')
     8 plt.title('Histogram of Voltage Amplitudes')
     9 plt.show()
```

Figure 6

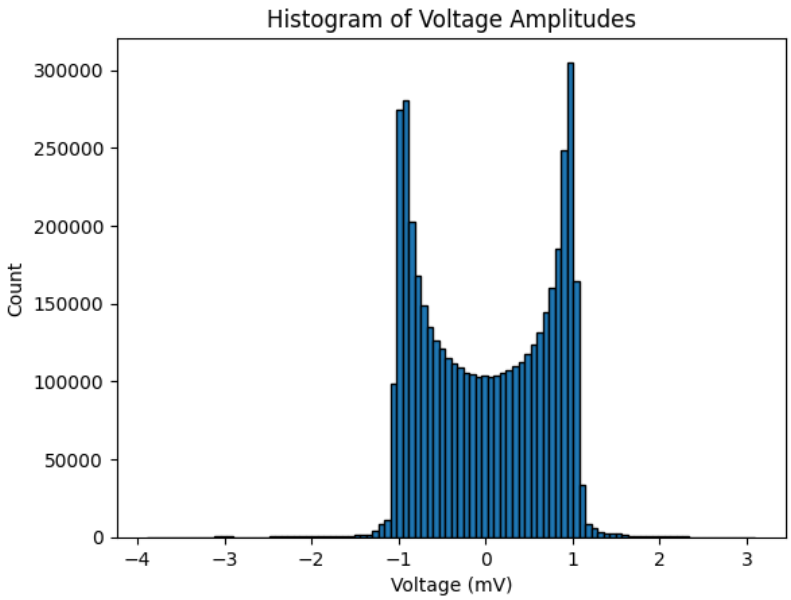

Figure 7

As expected, this diagram is a mirror. It can be said that small fluctuations are very visible in this graph. These fluctuations can be related to other neurons that we captured in the signal or it can be related to problems in the signal recording device.

Filtering the Data

In this part, we are going to design a 7th-order Butterworth high-pass filter with an off-cut frequency of 300 Hz using the signal library, and apply the designed filter to the data shown in the previous part using the signal.filtfilt() command. The code for the designed filter and its application to the given signal is shown in Figure 8. Also, the filtered and pre-filtered signal is shown in Figure 9.

Plot Unfiltered vs. Filtered Data Together

```
# Define filter parameters
highpass_freq = 300
filter_order = 7

# Calculate the Nyquist frequency
nyquist_freq = sampling_rate / 2

# Calculate the highpass cutoff frequency as a fraction of the Nyquist frequency
highpass_cutoff = highpass_freq / nyquist_freq

# Design the highpass Butterworth filter
b, a = signal.butter(filter_order, highpass_cutoff, btype='highpass')

# Apply the filter to the data using the filtfilt function
filtered_voltage = signal.filtfilt(b, a, voltage)

# Create a time axis based on the number of samples and sampling rate
time = np.arange(len(voltage)) / sampling_rate

plt.figure(figsize=(40,2))
# Plot the unfiltered data
plt.plot(time, voltage, label='Unfiltered')
# Plot the filtered data
plt.plot(time, filtered_voltage, label='Filtered')
plt.xlabel('Time (s)')
plt.ylabel('Voltage (mV)')
plt.title('Unfiltered vs. Filtered Data')
plt.legend()
plt.show()
```

Figure 8

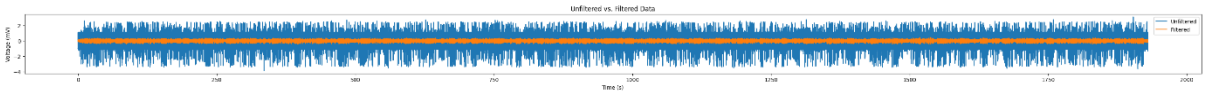

Figure 9

After we apply the high pass filter on the input signal, the obtained signal has less noise. As it is clear from the figure, the high frequencies caused by noise have been removed, so we obtained a clean signal and can use it for further analysis.

Detecting the Spike

e) In this part, we are going to obtain the threshold by using the following formulas for threshold and identify the spikes by comparing it with the filtered signal.

$$\sigma = \text{median}\left(\frac{|x|}{0.6745}\right), \qquad \theta = 5 * \sigma$$

First, we calculate the Threshold value for all the points of the given voltage range, and then by comparing all the points with the Threshold, we get the Peak points, after that, to display them, separate the 2ms after the peak and the 2ms before the peak and put them in the matrix We put spikes for display and the next parts. In Figure 10, the code for detecting spikes is written using derivative and threshold. Also, in Figure 11, all revealed spikes are displayed. It is clear that several different spikes had an effect on the generation of this signal, in other words, several neurons are involved in the production of these spikes.

```
# Calculate the highpass cutoff frequency as a fraction of the Nyquist frequency
highpass_cutoff = highpass_freq / nyquist_freq

# Design the highpass Butterworth filter
b, a = signal.butter(filter_order, highpass_cutoff, btype='highpass')

# Apply the filter to the data using the filtfilt function
filtered_voltage = signal.filtfilt(b, a, voltage)

# Calculate the voltage threshold (θ)
sigma_n = np.median(np.abs(filtered_voltage)) / 0.6745
threshold = 5 * sigma_n

print("Voltage Threshold (θ):", threshold)

# Extract peaks in the data
peaks, _ = find_peaks(filtered_voltage, height=0)  # Find peaks with positive height

# Select points above the threshold
spike_points = peaks[filtered_voltage[peaks] >= threshold]

# Define the time window for waveform extraction
window_size = int(fs * 0.002)  # 2ms before and after the peak
time_window = np.arange(-window_size, window_size + 1) / fs

# Extract waveforms for each detected spike
waveforms = []
for point in spike_points:
    waveform = filtered_voltage[point - window_size: point + window_size + 1]
    waveforms.append(waveform)

waveforms = np.array(waveforms)

# Plot the waveforms for each detected spike
plt.figure()
for waveform in waveforms:
    plt.plot(time_window, waveform, alpha=0.5)

plt.xlabel('Time (s)')
plt.ylabel('Voltage Amplitude')
plt.title('Waveforms of Detected Spikes')
plt.show()
```

Figure 10

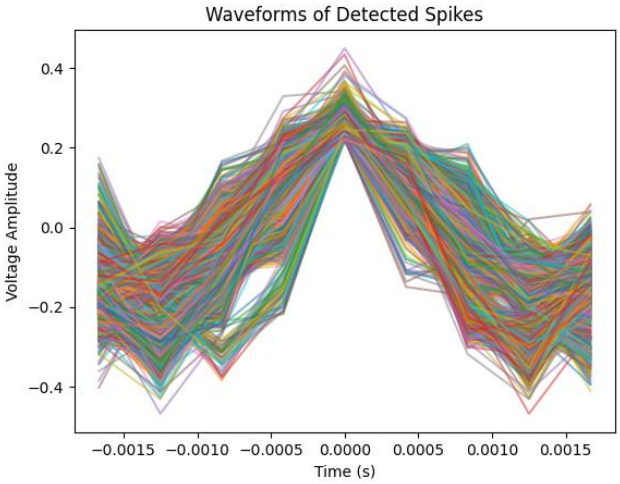

Figure 11

The threshold value is equal to 0.21. To detect the peaks, we count the places where the derivative has changed sign. After we have determined the peaks, we need to check whether this peak is greater than the threshold value using the threshold we found. If it was less, it is just a fluctuation and not significant. Therefore, we do not consider it.

Extracting Features

Using the scikit- learn library on the data, we want to find the PCAs. The number of features is from zero to seven, that is, there are 8 numbers. Therefore, it can be said that the maximum number of these axes is 8. Now we give these data to the function and print the score of each of the produced axes.

The output of the PCA algorithm determines the maximum variance of each of the 8 axes. We extract 3 of the first output values as features as shown in Figure12.

Feature extraction using PCA

```
# Apply PCA on the waveform matrix
pca = PCA(n_components=8)
waveform_pca = pca.fit_transform(waveforms)
print(pca.singular_values_)

PC1 = waveform_pca[:, 0]
PC2 = waveform_pca[:, 1]
PC3 = waveform_pca[:, 2]
PC = [PC1, PC2, PC3]

pca_waveform = PC

pca_waveform = np.array(PC)
pca_waveform = np.transpose(pca_waveform)
```

```
[8.5529941  5.61293533 3.28665284 2.82368678 2.57025087 2.49363466
 1.84777402 0.72617792]
```

Figure 12

Clustering the Spikes

In this part, we are going to use means-k to cluster the waveforms based on the characteristics of PC1, PC and PC3. For this purpose, we have used k=3, whose form is given below:

For k=3, the clustering of PC1 compared to PC2 is shown in Figure 13:

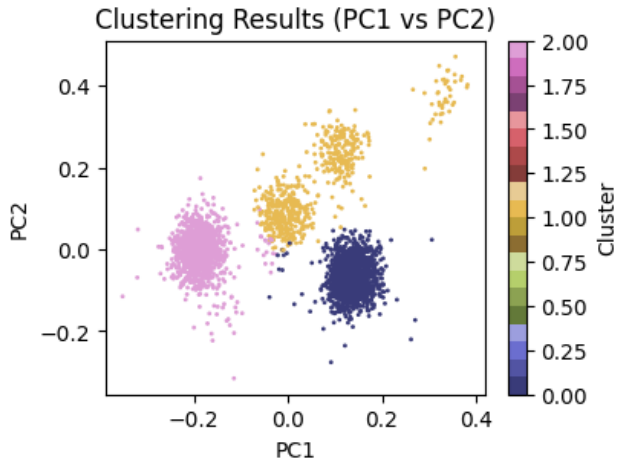

Figure 13

For k=3, the clustering of PC1 compared to PC3 is shown in Figure 14:

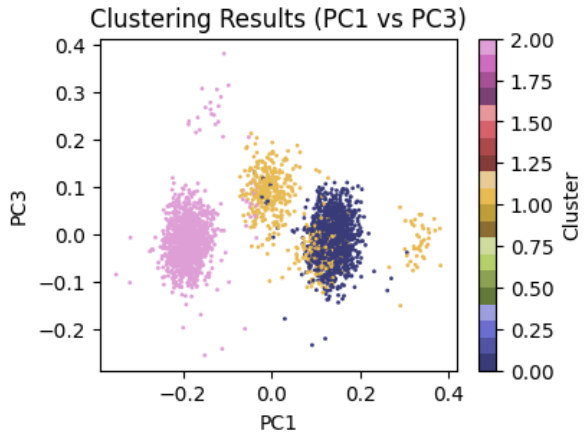

Figure 14

For k=3, the clustering of PC2 compared to PC3 is shown in Figure 15:

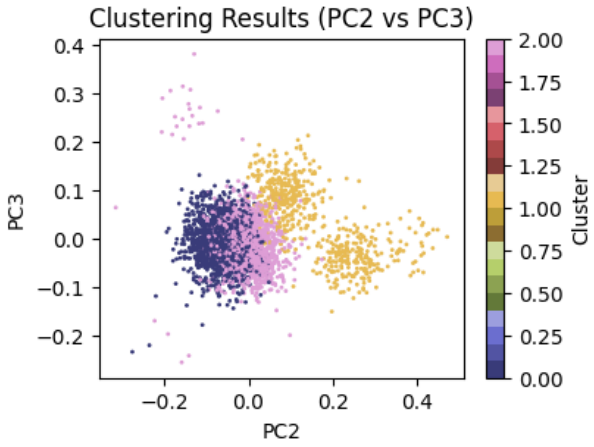

Figure 15

We took the data to the PCA space and selected the three axes that had the most points. It's time to do the clustering operation. We do the clustering using K means algorithm.Use bellow code in Figure 16 to cluster the PCA's feature.

Clustering using K-Means Clustering using PCA

```
[12]  1 # Perform K-Means clustering
      2 n_clusters = 3  # Number of clusters
      3 kmeans = KMeans(n_clusters=n_clusters, n_init="auto")
      4 kmeans.fit(pca_waveform)  # Use PC1, PC2, and PC3 as features for clustering
      5
      6 # Get the cluster labels for each waveform
      7 cluster_labels = kmeans.labels_
      8
      9 # Print the cluster labels
     10 print("Cluster Labels:", cluster_labels)

     Cluster Labels: [2 0 2 ... 0 2 0]
```

Figure 16

In the following, we draw the code and graphs related to each of the features (PCA) in each row, for each number of clusters in Figure 17 and 18. That is, the graphs in each column correspond to the binary combination of three features, and the graphs in each row correspond to the number of clusters.

Using the silhouette criterion, we draw the code and graph of the clustering performance in figures 19 and 20. As it is clear from the diagram of Figure 18 and Figure 20, the number of clusters of 3 in the first diagram is the best choice and the data are well clustered into 3 parts.

Plot the Result of clustering with different K's using PCA

```
[16]  1 # Repeat part (k) and (l) with different values of K for the clustering algorithm
      2 k = 8
      3 k_values = range(2, k)
      4 fig, axes = plt.subplots(k-2, 3, sharex=True, figsize=(4 * 4, (k-2) * 3))
      5 for i in k_values:
      6     kmeans = KMeans(n_clusters=i, random_state=k, n_init="auto").fit(waveform_pca)
      7     labels = kmeans.labels_
      8     axes[i-2, 0].scatter(waveform_pca[:,0], waveform_pca[:,1], c = labels, s=1, cmap='tab20b')
      9     axes[i-2, 0].set_title('Number of Clusters: ' + str(i))
     10     axes[i-2, 0].set_xlabel('PC1')
     11     axes[i-2, 0].set_ylabel('PC2')
     12
     13     axes[i-2, 1].scatter(waveform_pca[:,0], waveform_pca[:,2], c = labels, s=1, cmap='tab20b')
     14     axes[i-2, 1].set_title('Number of Clusters: ' + str(i))
     15     axes[i-2, 1].set_xlabel('PC1')
     16     axes[i-2, 1].set_ylabel('PC3')
     17
     18     axes[i-2, 2].scatter(waveform_pca[:,1], waveform_pca[:,2], c = labels, s=1, cmap='tab20b')
     19     axes[i-2, 2].set_title('Number of Clusters: ' + str(i))
     20     axes[i-2, 2].set_xlabel('PC2')
     21     axes[i-2, 2].set_ylabel('PC3')
     22
     23 # Add overall title and adjust layout
     24 fig.suptitle('KMeans Clustering Results', fontsize=14)
     25 fig.tight_layout()
```

Figure 17

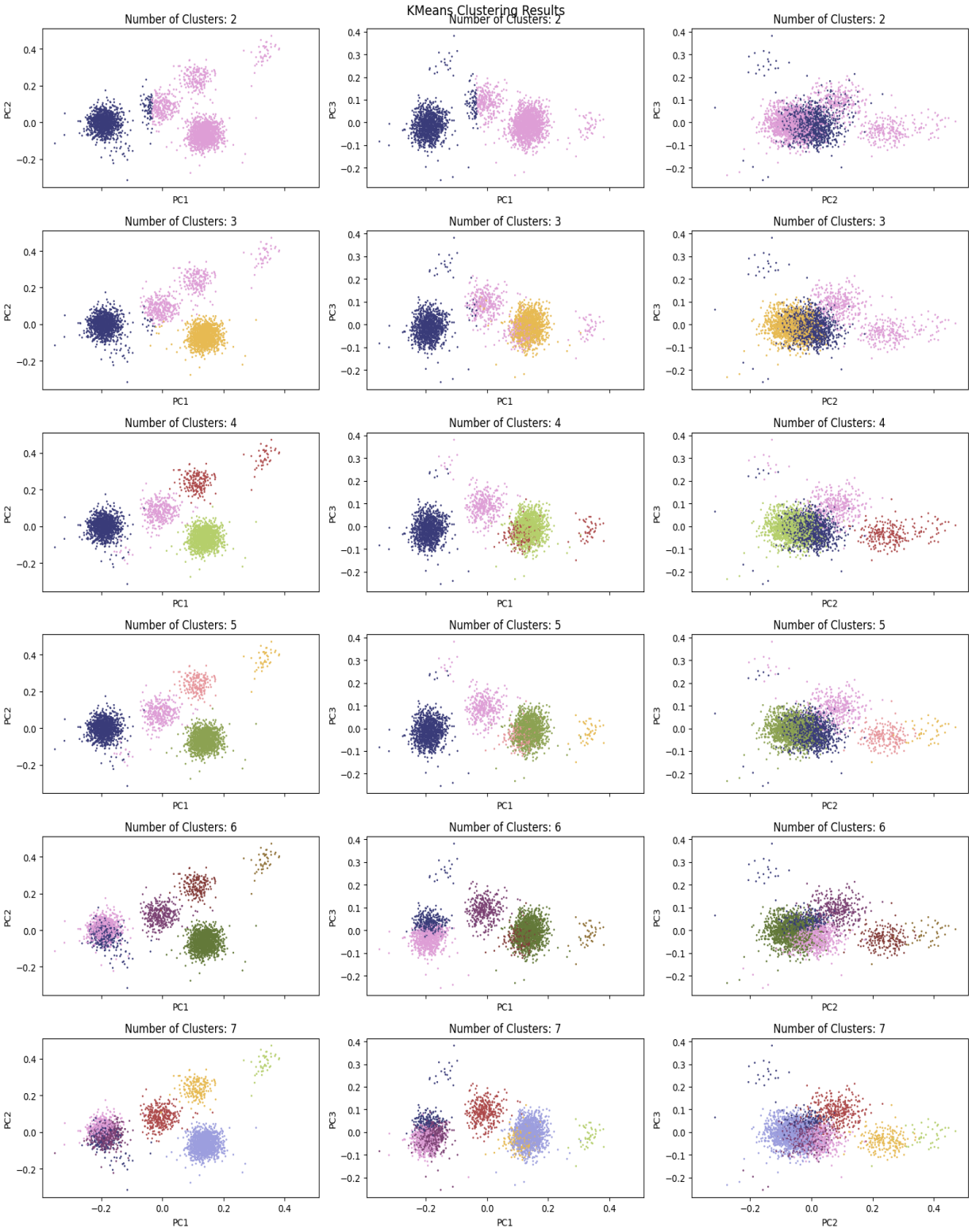
KMeans Clustering Results
Number of Clusters: 2
Number of Clusters: 2
Number of Clusters: 2
Number of Clusters: 3
Number of Clusters: 3
Number of Clusters: 3
Number of Clusters: 4
Number of Clusters: 4
Number of Clusters: 4
Number of Clusters: 5
Number of Clusters: 5
Number of Clusters: 5
Number of Clusters: 6
Number of Clusters: 6
Number of Clusters: 6
Number of Clusters: 7
Number of Clusters: 7
Number of Clusters: 7
PC1
PC2
PC3

Figure 18

Using silhouette_score to Find a K which was the best performance using PCA

```python
# Define a range of values for K
k_values = range(2, 8)

# Calculate silhouette scores for each value of K
silhouette_scores = []
for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=10, n_init="auto")
    labels = kmeans.fit_predict(waveform_pca)
    score = silhouette_score(waveform_pca, labels)
    silhouette_scores.append(score)

# Plot the silhouette scores for each value of K
import matplotlib.pyplot as plt

plt.plot(k_values, silhouette_scores)
plt.xlabel('Number of clusters (K)')
plt.ylabel('Silhouette score')
plt.show()
```

Figure 19

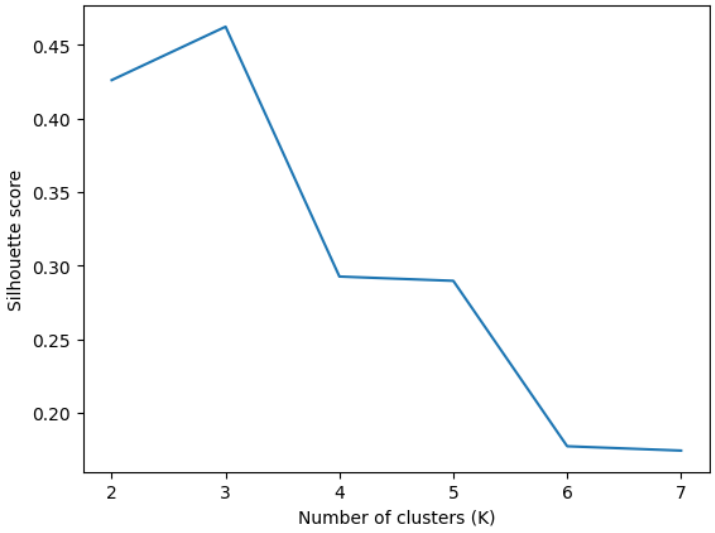

Figure 20

In Figure 21, we examine the performance of k-means clustering using 3 PCA features in spike detection using the given threshold. As we can see using the results, it has an acceptable performance.

Compute performance metrics of the K-means clustering by using PCA

```
[17]  1 # Compute performance metrics
      2 time_window = 0.001  # +/- 1ms
      3 true_positives = 0
      4 false_positives = 0
      5 false_negatives = 0
      6
      7 for spike_time in spike_idx:
      8     if np.any(np.abs(peaks - spike_time) <= time_window * fs):
      9         true_positives += 1
     10     else:
     11         false_negatives += 1
     12
     13 for spike_time in spikes:
     14     if np.all(np.abs(spike_time - spike_idx) > time_window * fs):
     15         false_positives += 1
     16
     17 precision = true_positives / (true_positives + false_positives)
     18 recall = true_positives / (true_positives + false_negatives)
     19 f1_score = 2 * (precision * recall) / (precision + recall)
     20
     21 print("Precision: %.3f" % precision)
     22 print("Recall: %.3f" % recall)
     23 print("F1-Score: %.3f" % f1_score)
     24
```

```
Precision: 0.772
Recall: 0.957
F1-Score: 0.855
```

Figure 21

New Threshold

In this part, we change the threshold and repeat all the previous steps.

When using a threshold equal to 90% of the magnitude of the previous threshold, an error occurs. The final number of detected spikes in this case is less than the acceptable number for finding PCA, so in this case the code returns an error. You can make the threshold a little smaller to avoid the error.

By setting the threshold with 80% of the magnitude of the previous threshold, a very small number of signals are recognized as spikes, and after clustering the following graphs, each row is for a different number of clusters, and each column is a combination of two of the three PCA features.

It seems that the smaller this number is, the more scattered the clusters will be, and therefore the results will not be very favorable.

$$\theta_{new} = 0.9 \times \max_{0 \le t \le T}(Xt)$$

First, we calculate the Threshold value for all the points of the given voltage range, and then by comparing all the points with the Threshold, we get the Peak points, after that, to display them, separate the 2ms after the peak and the 2ms before the peak and put them in the matrix. We put spikes for display and the next parts. In Figure 22, the code for detecting spikes is written using derivative and threshold. Also, in Figure 23, all revealed spikes are displayed. It is clear that several different spikes had an effect on the generation of this signal, in other words, several neurons are involved in the production of these spikes.

Filtering: Applying a Bandpass Filter Using New Threshold

```
[21]
# extract the voltage amplitude data from the structure
voltage = data['all_data_with_noise_and_line'][0,:]

# Define high-pass filter parameters
fs = 2400  # Sampling rate
f_high = 300  # High-pass cutoff frequency
order = 7  # Filter order

# Design the filter coefficients
b, a = signal.butter(order, f_high / (fs / 2), btype='highpass')

# Apply the filter to the data
filtered_data = signal.filtfilt(b, a, voltage)

# Find peaks in the data
diff_data = np.diff(filtered_data)
peaks = np.where((diff_data[:-1] > 0) & (diff_data[1:] < 0))[0] + 1

# Calculate the voltage threshold
new_theta = 0.8 * np.max(filtered_data)
p = filtered_data[peaks] >= new_theta

# Find spikes based on threshold
new_spikes = peaks[filtered_data[peaks] >= new_theta]
print(new_spikes)

# Extract waveform for each spike
window_size = int(0.004 * fs)  # 2ms window
new_waveforms = np.zeros((len(new_spikes), window_size))
for i, s in enumerate(new_spikes):
    new_waveform_start = max(s - window_size // 2, 0)
    new_waveform_end = min(s + window_size // 2, len(filtered_data))
    new_waveform = filtered_data[new_waveform_start:new_waveform_end]
    if len(new_waveform) < window_size:
        new_waveform = np.concatenate((np.zeros(window_size - len(new_waveform)), new_waveform))
    new_waveforms[i, :] = new_waveform

# Plot waveforms
plt.figure()
plt.plot(new_waveforms.T)
plt.xlabel('Time (samples)')
plt.ylabel('Amplitude (uV)')
plt.title('Waveforms of Detected Spikes')
plt.show()
new_waveforms = np.array(new_waveforms)
```

Figure 22

After applying the new threshold on the waveform data, we have the revealed spikes in Figure 23, which are less in number compared to the previous state, so our error increased compared to the previous state by changing the threshold, because the threshold level has increased and we have a lot of spikes.

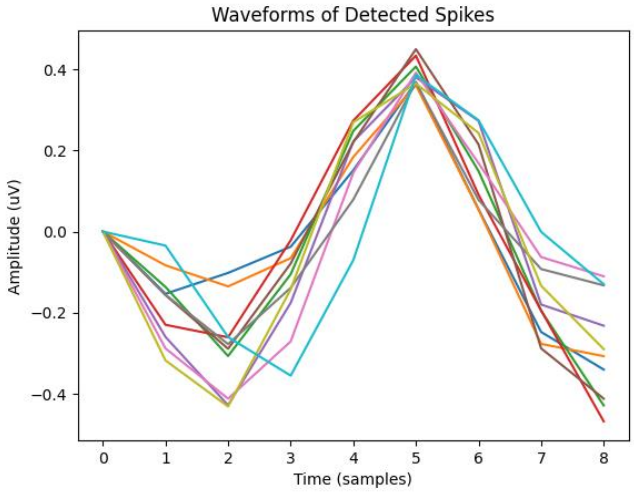

Figure 23

To detect the peaks, we count the places where the derivative has changed sign. After we have determined the peaks, we need to check whether this peak is greater than the threshold value using the threshold we found. If it was less, it is just a fluctuation and not significant. Therefore, we do not consider it.

Extracting Features with New Threshold

Using the scikit- learn library on the data, we want to find the PCAs. The number of features is from zero to seven, that is, there are 8 numbers. Therefore, it can be said that the maximum number of these axes is 8. Now we give these data to the function and print the score of each of the produced axes.

The output of the PCA algorithm determines the maximum variance of each of the 8 axes. We extract 3 of the first output values as features as shown in Figure24.

Using K-Means Clustering with New Threshold

```
[22]  1 # Apply PCA on the waveform matrix
      2 pca = PCA(n_components=8)
      3 new_waveform_pca = pca.fit_transform(new_waveforms)
      4 print(pca.singular_values_)
      5
      6 new_PC1 = new_waveform_pca[:, 0]
      7 new_PC2 = new_waveform_pca[:, 1]
      8 new_PC3 = new_waveform_pca[:, 2]
      9 new_PC = [new_PC1, new_PC2, new_PC3]
     10
     11 new_pca_waveform = new_PC
     12
     13 new_pca_waveform = np.array(new_PC)
     14 new_pca_waveform = np.transpose(new_pca_waveform)
```

```
[0.62258675 0.45924231 0.25186375 0.14795406 0.0889995  0.07161821
 0.06647049 0.01176512]
```

Figure 24

Clustering the Spikes with New Threshold

In this part, we are going to use means-k to cluster the waveforms based on the characteristics of PC1, PC and PC3. For this purpose, we have used k=3, whose form is given below:

For k=3, the clustering of PC1 compared to PC2 is shown in Figure 25:

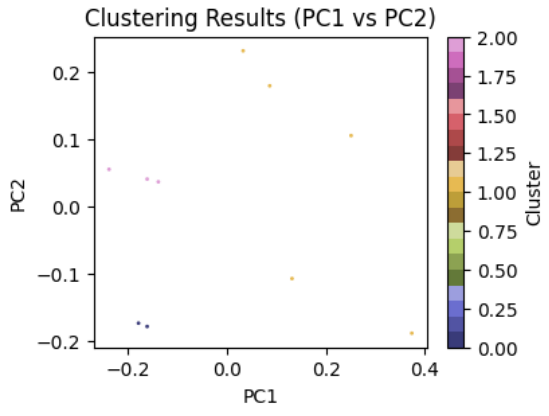

Figure 25

For k=3, the clustering of PC1 compared to PC3 is shown in Figure 26:

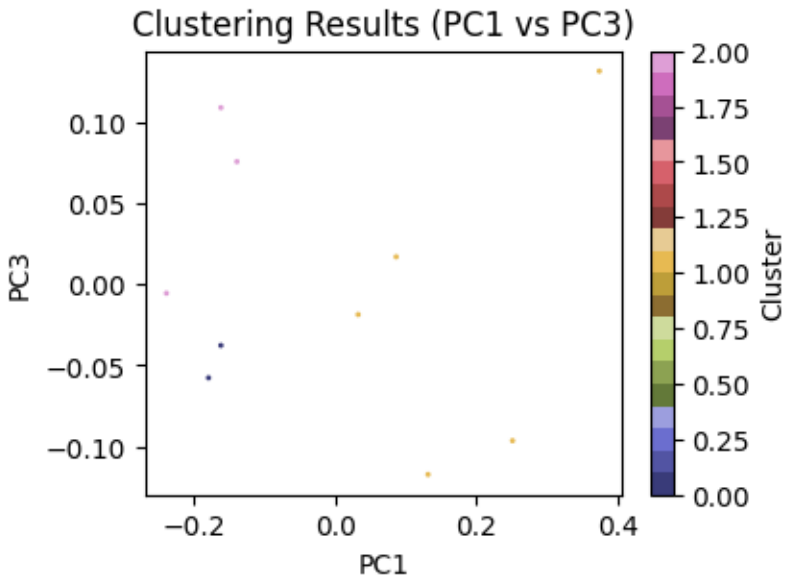

Figure 26

For k=3, the clustering of PC2 compared to PC3 is shown in Figure 27:

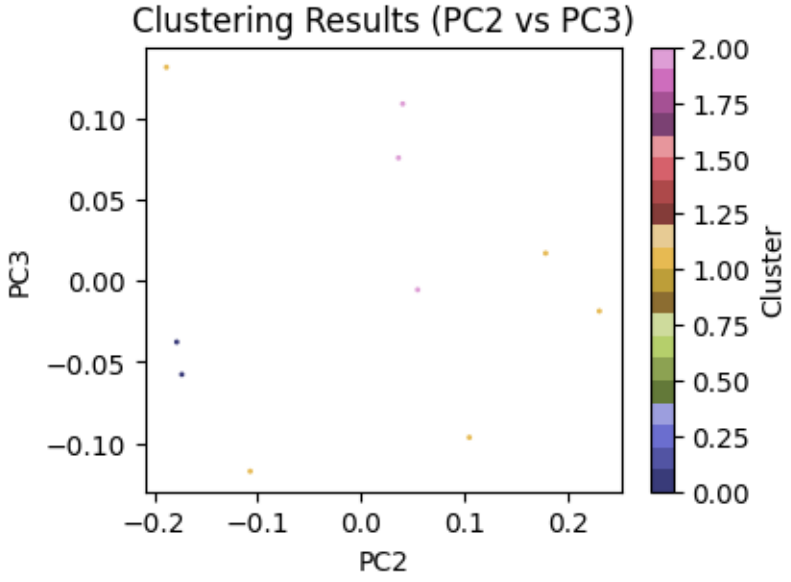

Figure 27

We took the data to the PCA space and selected the three axes that had the most points. It's time to do the clustering operation. We do the clustering using K means algorithm. Use bellow code in Figure 28 to cluster the PCA's feature.

```python
# Perform K-Means clustering
n_clusters = 3  # Number of clusters
kmeans = KMeans(n_clusters=n_clusters, n_init="auto")
kmeans.fit(new_pca_waveform)  # Use PC1, PC2, and PC3 as features for clustering

# Get the cluster labels for each waveform
new_cluster_labels = kmeans.labels_

# Print the cluster labels
print("Cluster Labels:", new_cluster_labels)
```

```
Cluster Labels: [0 0 2 2 1 2 1 1 1 1]
```

Figure 28

In the following, we draw the code and graphs related to each of the features (PCA) in each row, for each number of clusters in Figure 29 and 30. That is, the graphs in each column correspond to the binary combination of three features, and the graphs in each row correspond to the number of clusters.

Using the silhouette criterion, we draw the code and graph of the clustering performance in figures 31 and 32. As it is clear from the diagram of Figure 30 and Figure 32, the number of clusters of 5 in the first diagram is the best choice and the data are well clustered into 3 parts.

Plot the Result of clustering with different K's with new threshold

```python
[28]
# Repeat part (k) and (l) with different values of K for the clustering algorithm
k = 8
k_values = range(2, k)
fig, axes = plt.subplots(k-2, 3, sharex=True, figsize=(4 * 4, (k-2) * 3))
for i in k_values:
    kmeans = KMeans(n_clusters=i, random_state=k, n_init="auto").fit(new_waveform_pca)
    labels = kmeans.labels_
    axes[i-2, 0].scatter(new_waveform_pca[:,0], new_waveform_pca[:,1], c = new_labels, s=1, cmap='tab20b')
    axes[i-2, 0].set_title('Number of Clusters: ' + str(i))
    axes[i-2, 0].set_xlabel('PC1')
    axes[i-2, 0].set_ylabel('PC2')

    axes[i-2, 1].scatter(new_waveform_pca[:,0], new_waveform_pca[:,2], c = new_labels, s=1, cmap='tab20b')
    axes[i-2, 1].set_title('Number of Clusters: ' + str(i))
    axes[i-2, 1].set_xlabel('PC1')
    axes[i-2, 1].set_ylabel('PC3')

    axes[i-2, 2].scatter(new_waveform_pca[:,1], new_waveform_pca[:,2], c = new_labels, s=1, cmap='tab20b')
    axes[i-2, 2].set_title('Number of Clusters: ' + str(i))
    axes[i-2, 2].set_xlabel('PC2')
    axes[i-2, 2].set_ylabel('PC3')

# Add overall title and adjust layout
fig.suptitle('KMeans Clustering Results', fontsize=14)
fig.tight_layout()
```

Figure 29

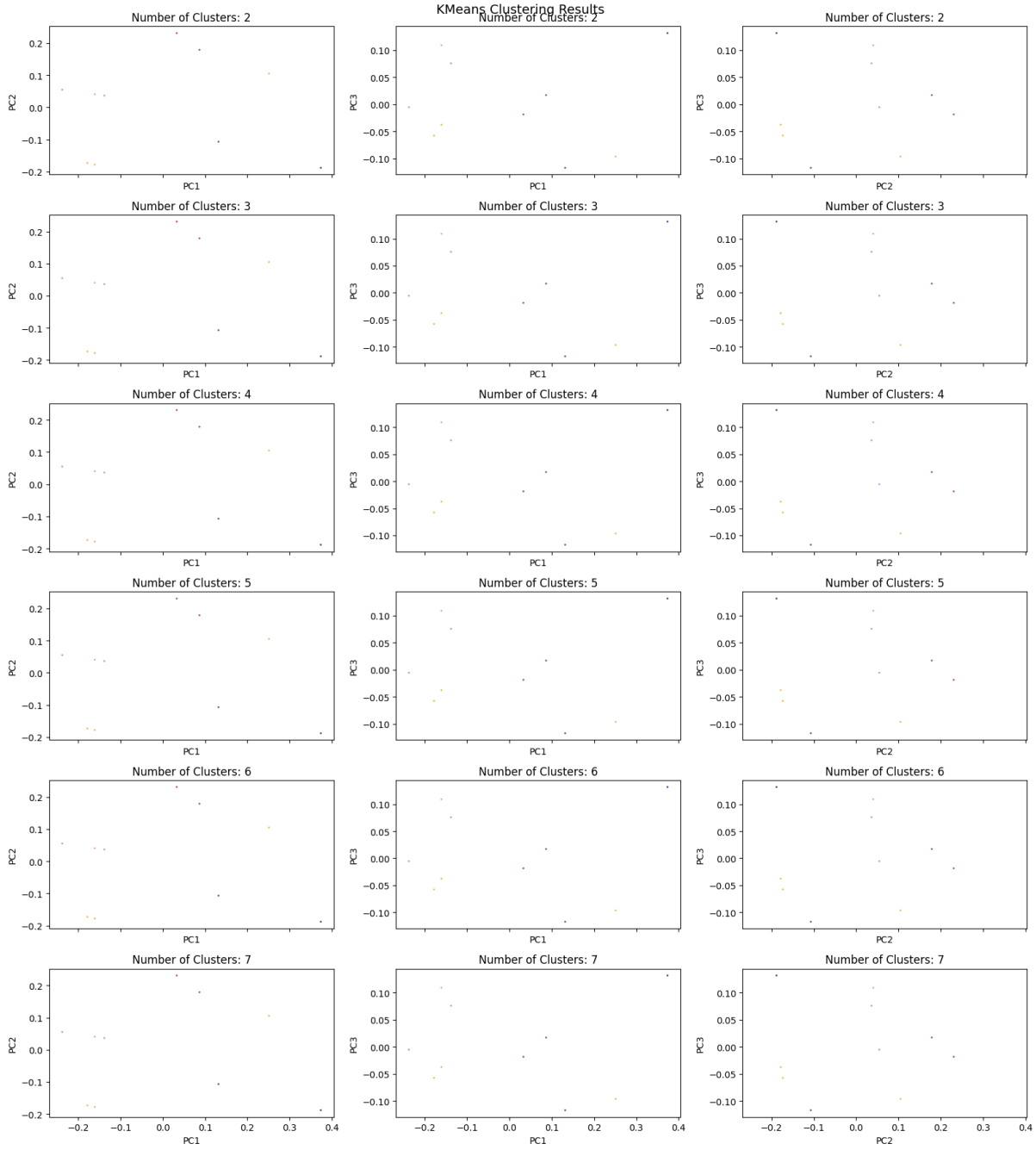
KMeans Clustering Results
Number of Clusters: 2
Number of Clusters: 2
Number of Clusters: 2
Number of Clusters: 3
Number of Clusters: 3
Number of Clusters: 3
Number of Clusters: 4
Number of Clusters: 4
Number of Clusters: 4
Number of Clusters: 5
Number of Clusters: 5
Number of Clusters: 5
Number of Clusters: 6
Number of Clusters: 6
Number of Clusters: 6
Number of Clusters: 7
Number of Clusters: 7
Number of Clusters: 7
PC1
PC2
PC3

Figure 30

Using silhouette_score to Find a K which was the best performance using new threshold

```
[27]  1 # Define a range of values for K
      2 k_values = range(2, 8)
      3
      4 # Calculate silhouette scores for each value of K
      5 new_silhouette_scores = []
      6 for k in k_values:
      7     kmeans = KMeans(n_clusters=k, random_state=10, n_init="auto")
      8     new_labels = kmeans.fit_predict(new_waveform_pca)
      9     score = silhouette_score(new_waveform_pca, new_labels)
     10     new_silhouette_scores.append(score)
     11
     12 # Plot the silhouette scores for each value of K
     13 plt.plot(k_values, new_silhouette_scores)
     14 plt.xlabel('Number of clusters (K)')
     15 plt.ylabel('Silhouette score')
     16 plt.show()
```

Figure 31

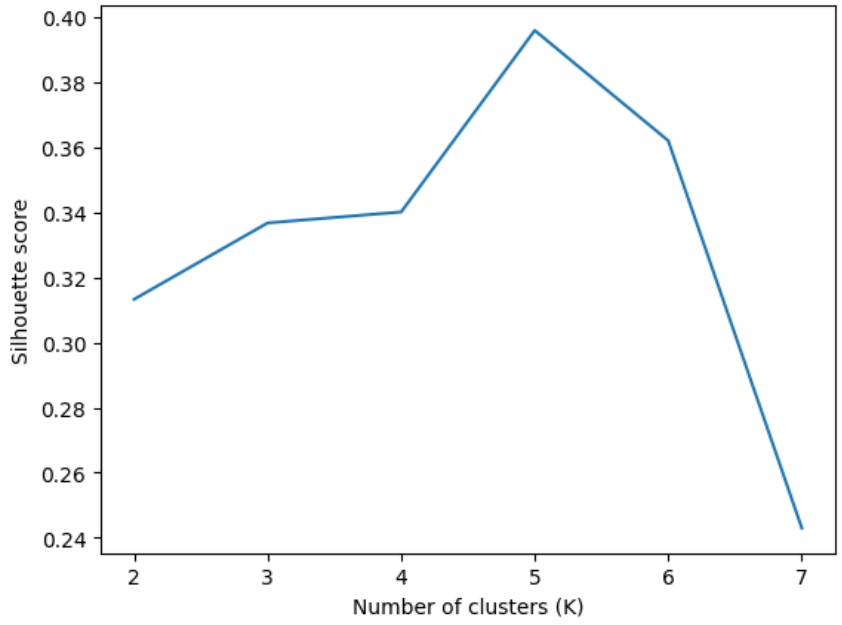

Figure 32

In Figure 33, we examine the performance of k-means clustering using 3 PCA features in spike detection using the given threshold. As we can see using the results, it has an acceptable performance.

Compute performance metrics of the K-means clustering for new threshold

```
# Compute performance metrics
time_window = 0.001  # +/- 1ms
true_positives = 0
false_positives = 0
false_negatives = 0

for spike_time in spike_idx:
    if np.any(np.abs(peaks - spike_time) <= time_window * fs):
        true_positives += 1
    else:
        false_negatives += 1

for spike_time in new_spikes:
    if np.all(np.abs(spike_time - spike_idx) > time_window * fs):
        false_positives += 1

precision = true_positives / (true_positives + false_positives)
recall = true_positives / (true_positives + false_negatives)
f1_score = 2 * (precision * recall) / (precision + recall)

print("Precision: %.3f" % precision)
print("Recall: %.3f" % recall)
print("F1-Score: %.3f" % f1_score)

```

```
Precision: 0.999
Recall: 0.957
F1-Score: 0.978
```

Figure 33

Precision measures the proportion of correctly detected spikes among all the detected spikes. A precision of 0.999 indicates that the vast majority of the detected spikes are true positives, meaning there are very few false positives.

Recall, also known as sensitivity or true positive rate, measures the proportion of correctly detected spikes among all the true spikes. A recall of 0.957 indicates that the pipeline is able to capture a high percentage of the true spikes, with only a small number of false negatives.

The F1-score is the harmonic mean of precision and recall, providing a balanced measure of the overall performance of the pipeline. An F1-score of 0.978 indicates that the pipeline achieves a good balance between precision and recall.

Based on these metrics, the spike sorting pipeline appears to perform well, with a high precision, recall, and F1-score. It suggests that the pipeline is effective in accurately detecting and classifying spikes in the data.

It's important to note that the evaluation metrics depend on the chosen time window of +/- 1ms. Adjusting the time window may impact the performance metrics, so it's essential to choose an appropriate time window based on the characteristics of the data and the specific requirements of the spike sorting task.

Feature Extraction using t-SNE

We explore the use of t-SNE algorithm as an alternative to PCA for feature extraction in the spike-sorting pipeline. We compare the results obtained from t-SNE with those obtained from PCA and analyze if there is any improvement in the performance of the pipeline.

Instead of using PCA, we apply the t-SNE algorithm to the waveforms matrix. This algorithm reduces the dimensionality of the data while preserving the local structure of the samples.

Implementation:

In Python, we can utilize the sklearn.manifold.TSNE class to perform t-SNE. After fitting the t-SNE model to the waveforms, we obtain three t-SNE components, denoted as tsne_1, tsne_2, and tsne_3**.** We extract 3 of the first output values as features as shown in Figure34.

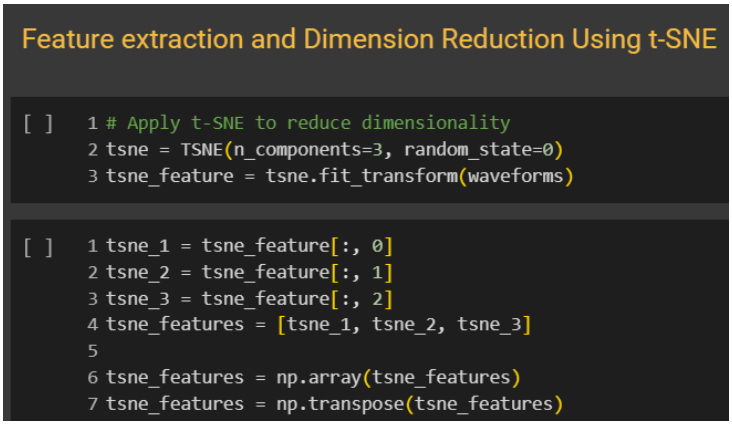

Figure 34

Clustering:

Next, we proceed with clustering the waveforms based on the t-SNE features obtained. We use the K-Means algorithm to cluster the waveforms into different groups. The number of clusters (K) can be chosen based on the characteristics of the data and the desired level of granularity.

Using the silhouette criterion, we draw the code and graph of the clustering performance in figures 35 and 36. As it is clear from the diagram of Figure 36 and Figure 38, the number of clusters of 3 in the first diagram is the best choice and the data are well clustered into 3 parts.

Using silhouette_score to Find a K which was the best performance for t-SNE algorithm

```
[ ]  1 # Define a range of values for K
     2 k_values = range(2, 8)
     3
     4 # Calculate silhouette scores for each value of K
     5 silhouette_scores = []
     6 for k in k_values:
     7     kmeans = KMeans(n_clusters=k, random_state=10, n_init="auto")
     8     labels = kmeans.fit_predict(tsne_features)
     9     score = silhouette_score(tsne_features, labels)
    10     silhouette_scores.append(score)
    11
    12 # Plot the silhouette scores for each value of K
    13 plt.plot(k_values, silhouette_scores)
    14 plt.xlabel('Number of clusters (K)')
    15 plt.ylabel('Silhouette score')
    16 plt.show()
```

Figure 35

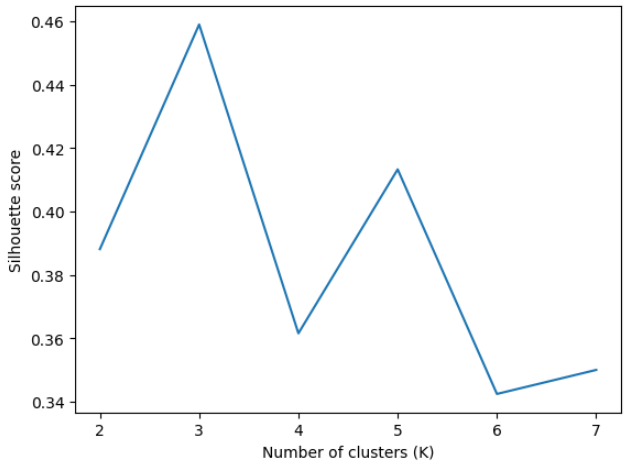

Figure 36

Visualization and Analysis:

We plot scatter plots of the t-SNE components to visualize the clusters and their separation in the reduced-dimensional space. Comparing the results with those obtained from PCA, we observe there is a lot of improvement in the clustering performance and separation of the spike waveforms.

Plot the Result of clustering with different K's using t-SNE algorithm

```
[ ]
# Repeat part (k) and (l) with different values of K for the clustering algorithm
k = 8
k_values = range(2, k)
fig, axes = plt.subplots(k-2, 3, sharex=True, figsize=(4 * 4, (k-2) * 3))
for i in k_values:
    kmeans = KMeans(n_clusters=i, random_state=k, n_init="auto").fit(tsne_features)
    labels = kmeans.labels_
    axes[i-2, 0].scatter(tsne_feature[:,0], tsne_features[:,1], c = labels, s=1, cmap='tab20b')
    axes[i-2, 0].set_title('Number of Clusters: ' + str(i))

    axes[i-2, 1].scatter(tsne_feature[:,0], tsne_features[:,2], c = labels, s=1, cmap='tab20b')
    axes[i-2, 1].set_title('Number of Clusters: ' + str(i))

    axes[i-2, 2].scatter(tsne_feature[:,1], tsne_features[:,2], c = labels, s=1, cmap='tab20b')
    axes[i-2, 2].set_title('Number of Clusters: ' + str(i))


# Add overall title and adjust layout
fig.suptitle('KMeans Clustering Results', fontsize=14)
fig.tight_layout()
```

Figure 37

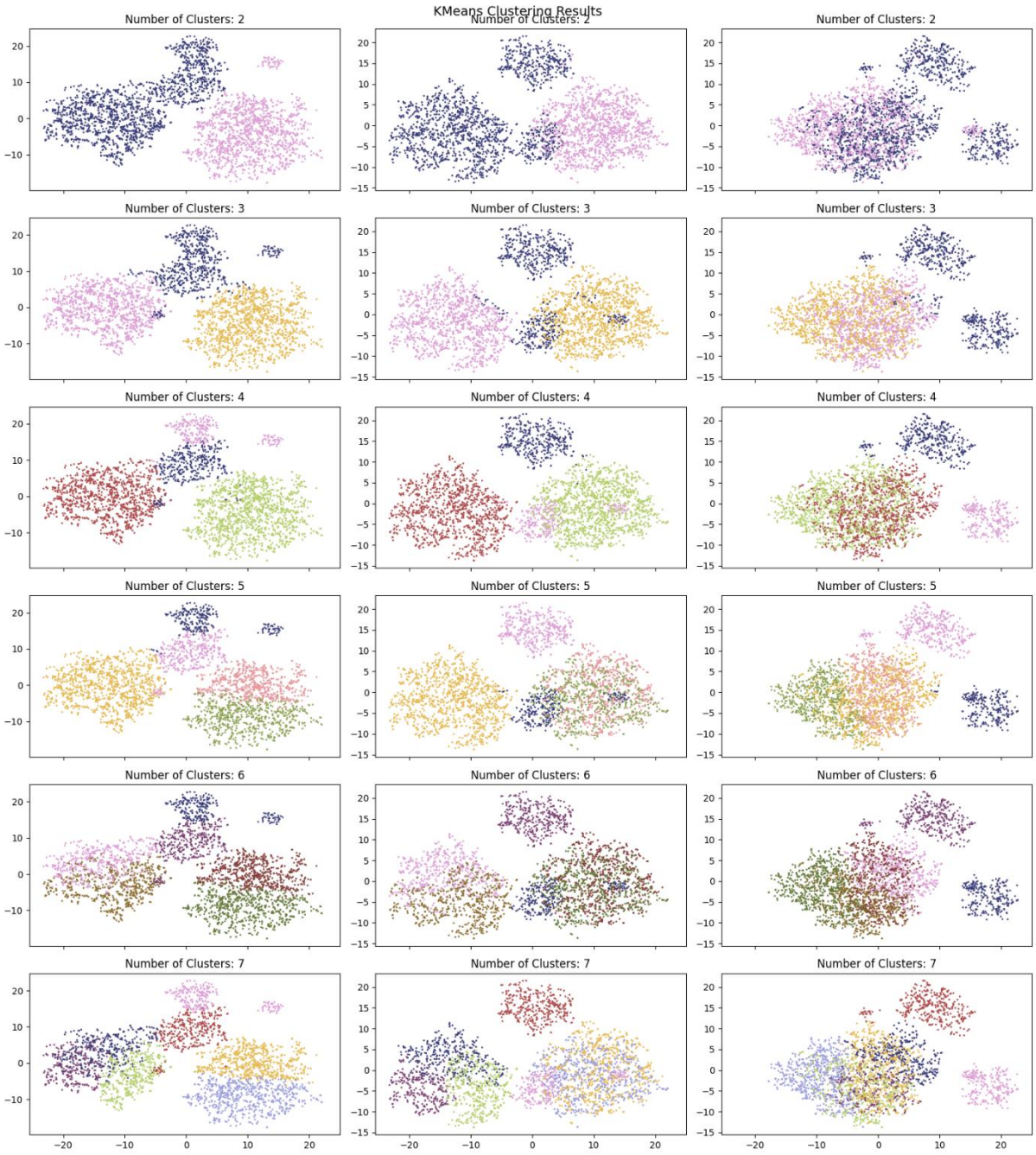

Figure 38

Conclusion:

In conclusion, by incorporating the t-SNE algorithm into the spike sorting pipeline, we can potentially achieve improved performance compared to using PCA for feature extraction. The t-SNE algorithm can capture more intricate nonlinear relationships among the spike waveforms, leading to enhanced separation and clustering of the spikes in the reduced-dimensional space.

However, the actual improvement in results may vary depending on the specific dataset and the parameter settings of t-SNE. Therefore, thorough experimentation and evaluation are crucial to determine the effectiveness of t-SNE in spike sorting tasks.

2. Spike Sorting with ROSS

Introduction:

Analyzing extracellular data from neuronal recordings involves the crucial step of spike sorting, which aims to differentiate between different neuron activities. Manual spike sorting can be time-consuming and laborious, especially for studies with multiple recording sessions. To address this issue, various software and toolboxes have been developed, one of which is ROSS (Rapid Offline Spike Sorting). ROSS is a MATLAB-based spike sorting software that offers both automatic and manual spike sorting capabilities, along with visualization tools to aid in the analysis process.

Detection:

The first step in spike sorting is activity detection. ROSS provides a feature to extract spikes from the bandpass-filtered signal. Spikes are detected based on activities crossing a threshold, which is calculated using the estimated noise power. Users can adjust settings such as filter type and thresholding method for accurate detection. The detected spikes and their occurrence times will be saved as a spike_extracellular_detection_part_2.mat file which was existed in the zip-file.

Action Steps:

- Load the raw extracellular data.
- Adjust the filtering and thresholding settings.
- Click the "Start Detection" button to visualize the detection results in a PCA plot, being showed in Figure 39.

By using Detection Spike, the diagram related to the histogram drawn was from the top view and in two dimensions, which we could look at from a three-dimensional angle using d3 rotate. Below is the picture of the diagram related to it.

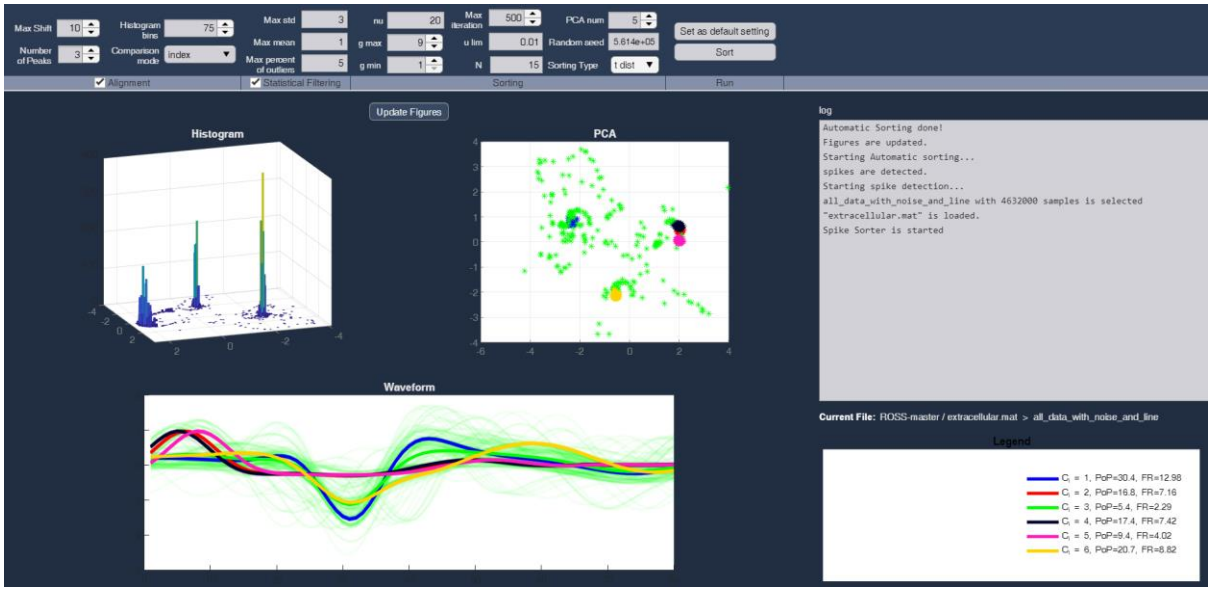

Figure 39

Auto Sorting:

Auto sorting is a clustering method used to automatically assign each spike to the corresponding neuron. ROSS utilizes a combination of techniques including mixture of t-distributions, Gaussian Mixture Models (GMM), k-means, and template matching for clustering. It also provides statistical filtering for noise removal and alignment methods. The output of this step is cluster indices for each spike waveform, which can be saved as a sort_res_extracellular_sorting

_part_2.mat file.

Action Steps:

- Use the auto-sorting feature in ROSS.
- Observe the results in PCA and waveform plots.
- Determine the number of clusters present in the data.

Manual Sorting:

Manual sorting in ROSS allows researchers to modify the auto sorting output to improve the results. Various tools are available, including Merge, Delete, Resort, Denoise, and Manual grouping or deleting in PCA domain of clusters. Additionally, researchers can add arbitrary comments to tag neurons for further analysis. The cluster indices, along with the neuron tags, can be saved as a MAT file.

Action Steps:

- Switch to the 'Manual Sorting' tab in ROSS.
- Select the 'Denoise' option and adjust the data plot percentage and denoising threshold to 85.
- Choose a cluster and apply the automatic sorting process to it.
- Observe if the selected cluster divides into new clusters and determine the number of resulting clusters.

The bellow graphs show how much of the waves were removed with a threshold of 85%. The right side shows the spikes that are recognized as noise, which are overwritten after the Save action is performed to re-store the data.

Manual Sorting for one cluster is shown in Figure 40.

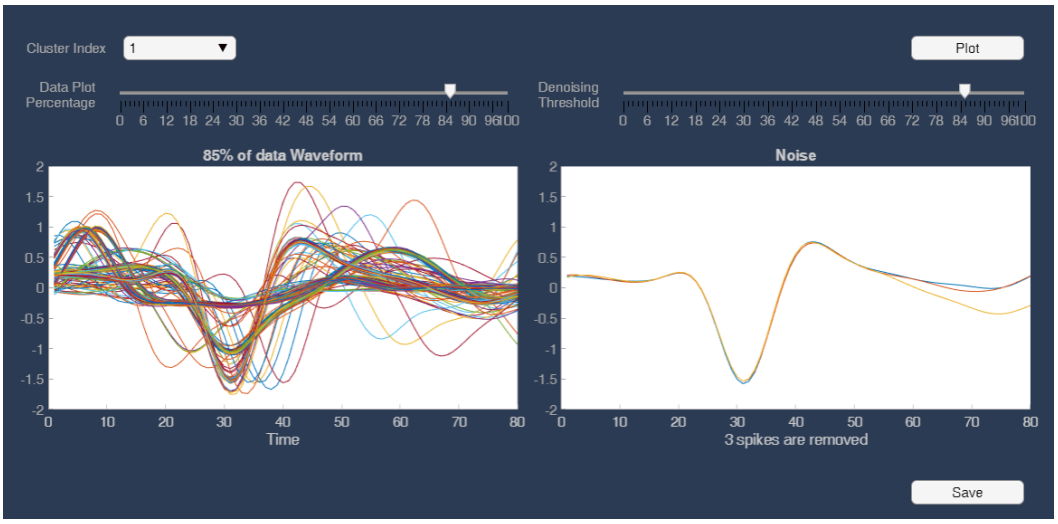

Figure 40

Manual Sorting for four cluster is shown in Figure 41.

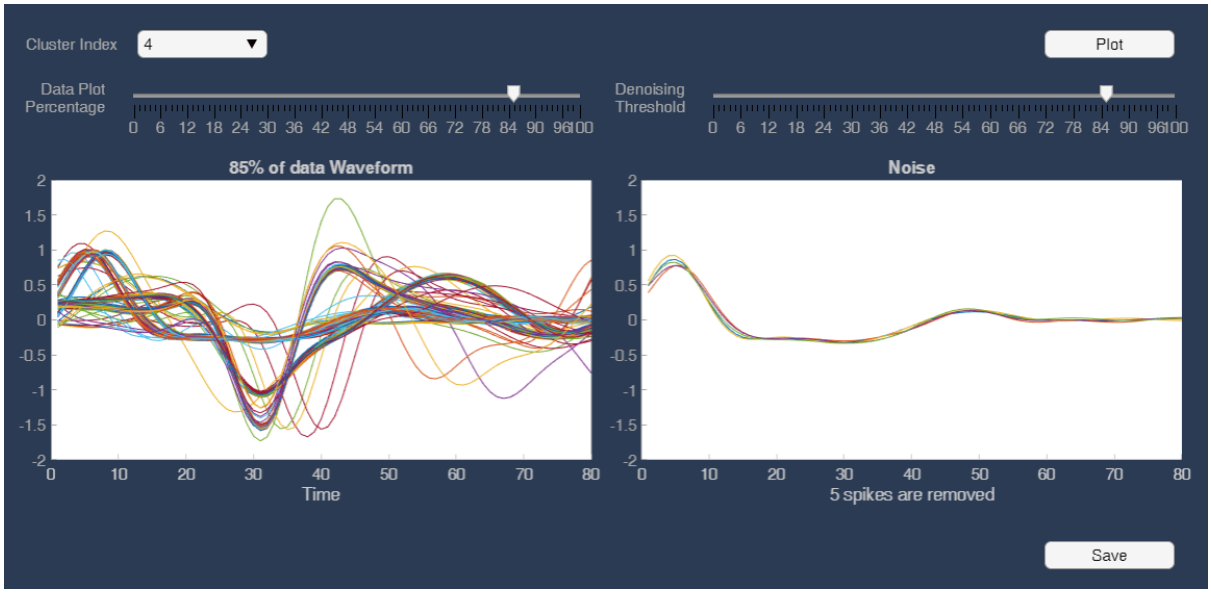

Figure 41

Manual Sorting for six cluster is shown in Figure 42.

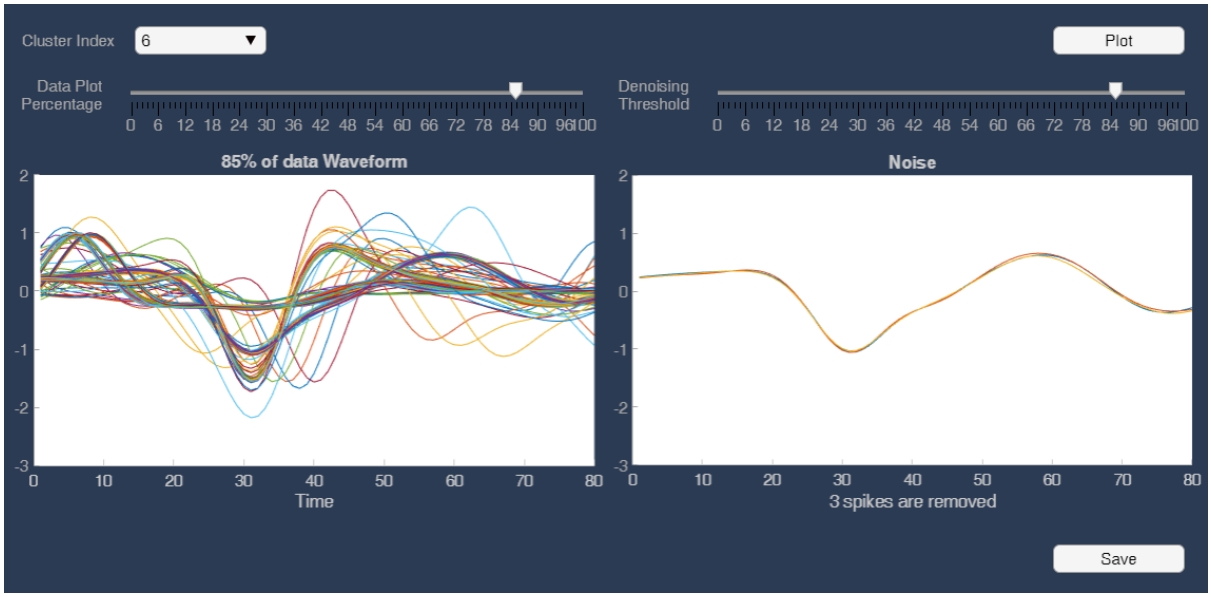

Figure 42

Visualization:

ROSS provides a range of visualization tools to facilitate a better understanding of the analyzed extracellular data. These tools include Inter-spike interval, Neuron lifetime, Waveforms, 3D plot, and PCA domain plots. Additionally, it allows tracking detected spikes on the raw data. Each neuron is assigned a unique color across all visualizations in the software.

Recommended Visualization Tasks:

- Plot the waveforms of different clusters in separate plots and compare them, are shown in Figure 43.
- Use a 3D plot to display the first two PCA components over time, is shown in Figure 44.
- Plot the entire raw data with the detected spikes and compare waveform shapes of different clusters, is shown in Figure 45.

We show the diagram related to the waves of each cluster separately in Figure 43. The interval of domain changes for green cluster is more than other clusters. The black color cluster is the cluster with the least ups and downs, the range distance in each peak and valley is very small, and its graph is very smooth, so this graph does not have a sharp peak. That is, we have mostly black clusters in a straight line that we can see in the middle part of that valley. The blue and red clusters both have relatively the same interval, but in the green cluster we see 3 peaks on average in our wave, while in the red cluster it is not the same and after leaving the valley, we see a shoulder. The pink cluster can be considered another cluster, which is not the same as any other cluster, but does not follow a specific pattern.

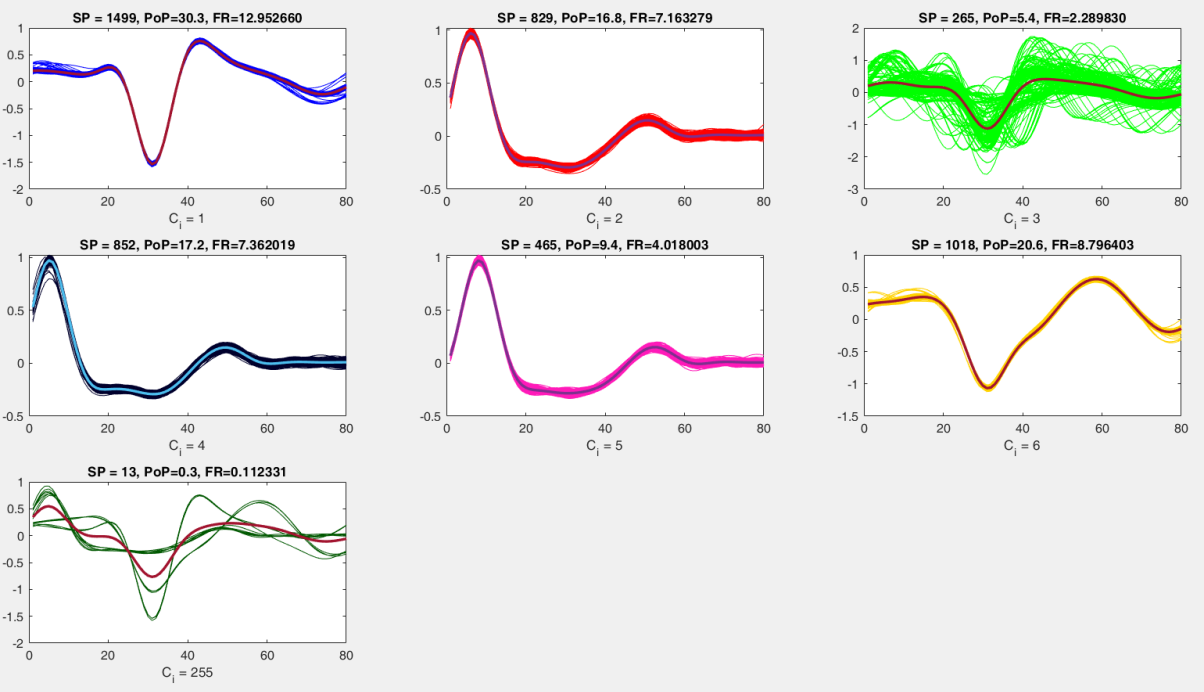

Figure 43

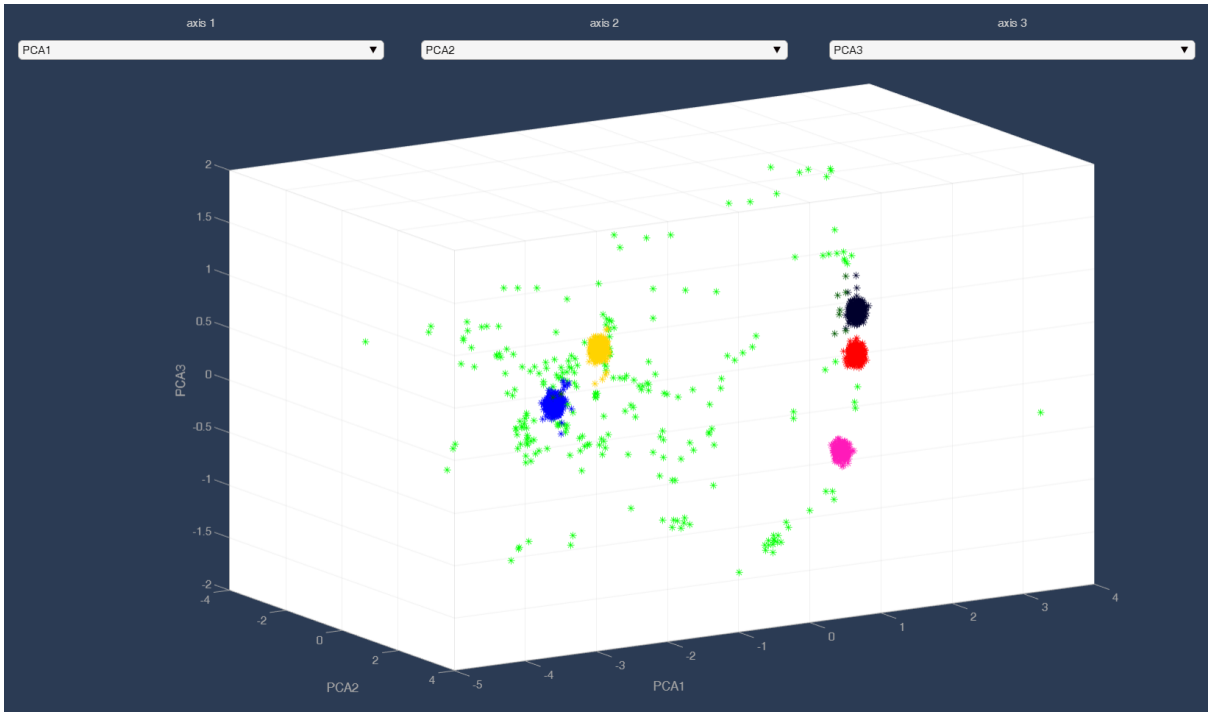

Figure 44

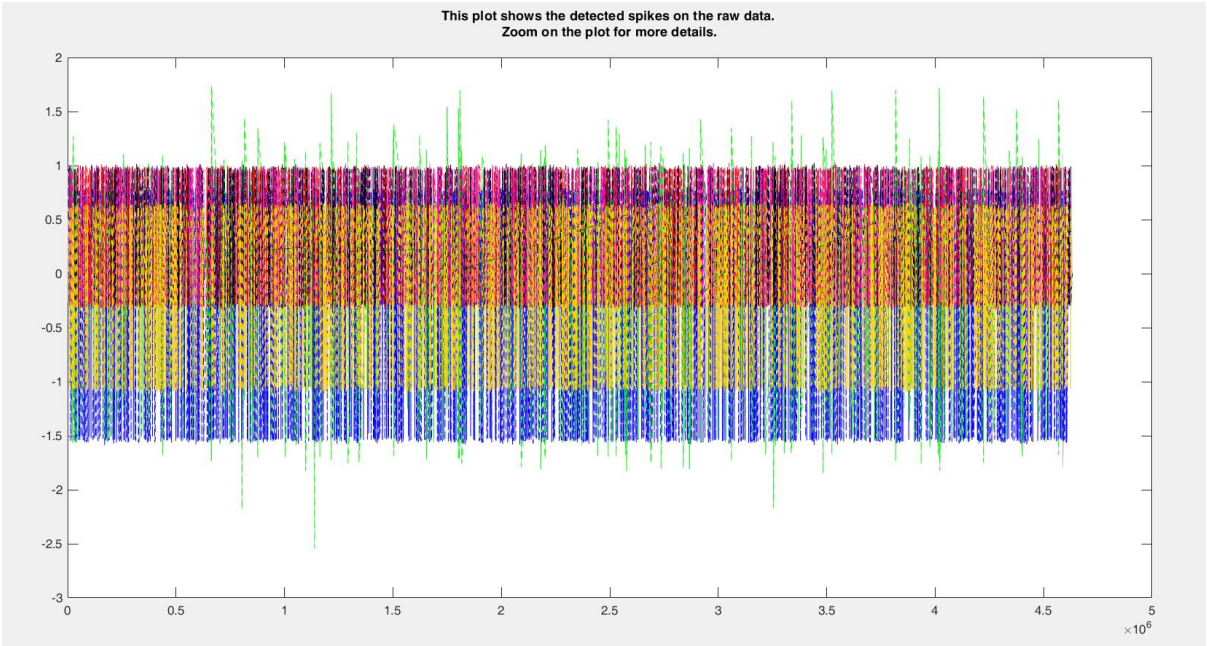

Figure 45

In the following, the graph of the total data is given in Figure 45. As we can see, most of the clusters are not easily distinguishable from each other. Only the blue clusters can be separated and recognized by the eye in such a way that the waves that have a very low spike are placed in the blue cluster. Other clusters are so intertwined that they cannot be distinguished.

3. Analysis of Single Neuron Activity in Response to Face and Non-Face Stimuli:

Introduction:

Understanding encode and process individual neurons information is a fundamental part in neuroscience. In this study, we explore the use of two commonly used metrics, d-prime and mutual information, to analyze the information processing capabilities of single neurons in response to face and non-face stimuli. We investigate the limitations and biases of mutual information and apply it to estimate the amount of face information encoded by each neuron's response. Furthermore, we compare the timing of information emergence in neurons with the onset of the rise in firing rate plots to gain insights into the temporal dynamics of information processing.

Mutual information is a useful metric for analyzing single neuron activity, but there are some limitations and sources of bias that can impact its interpretation in real-world practice. One limitation is that mutual information is affected by the amount of noise present in the data. If there is a lot of noise in the data, then mutual information may underestimate the amount of information that a neuron is actually encoding. Additionally, mutual information can be biased by the choice of bin size used to group the firing rate data. If the bin size is too large, then some information may be lost, while if the bin size is too small, then noise may be amplified.

To apply mutual information to the dataset of single-neuron activity, we can use the time window-averaged signal to calculate the firing rate for each neuron in response to face and non-face stimuli. Then, we can apply mutual information to each time bin's firing rate and estimate the amount of face information in each neuron's response. Finally, we can average the mutual information across neurons and plot the results, including the mean and standard error of the mean.

Visualization

Using array.numpy, you can load the court on the Google server and place it in the label and data variables, which are shown in Figure 46.

Load the Data

```
[ ]  1 # Load the time series data and labels from numpy files
     2 data = np.load('/content/drive/MyDrive/Cognetive/HW_2/assignment2-face-data.npy')
     3 labels = np.load('/content/drive/MyDrive/Cognetive/HW_2/assignment2-face-data-labels.npy')
     4 print(labels.shape)
```

Figure 46

To reduce data noise and calculate firing rate, we move an average window along the time axis, with length 50 samples and step 1 using convolution. Then we plot the time series of each neuron's response in time. We classify observations based on 3 types of stimuli, which is shown, in Figure 47.

Moving Average through the Time

```
[ ]  1 # Calculate firing rate using a moving average window of length 50
     2 window_length = 50
     3 str_ = 1
     4
     5 Moving_average = np.zeros((data.shape[0], data.shape[1], (data.shape[2] - window_length + 1)))
     6 for i in range(data.shape[0]):
     7   for j in range(data.shape[1]):
     8     Moving_average[i,j] = np.convolve(data[i,j,:], np.ones(window_length)/window_length, mode='valid')[::str_]
```

Figure 47

To visualize Moving Average (Convolution filter), applied the bellow code in Figure 48, and show the best observation in the basis of stimulus in Figure 49.

Visualization Moving Average through the Time

```
[ ]  1 # Plot the time series of each neuron's response to face and non-face stimuli
     2 fig, axs = plt.subplots(nrows=6, ncols=1, figsize=(8,12), sharex=True)
     3 for i in range(6):
     4     face_trials = np.where(labels == 0)[0]
     5     nonface_trials = np.where(labels == 2)[0]
     6     monkey_trials = np.where(labels == 1)[0]
     7     axs[i].plot(np.mean(Moving_average[face_trials,i,:], axis=0), label='face')
     8     axs[i].plot(np.mean(Moving_average[nonface_trials,i,:], axis=0), label='non-face')
     9     axs[i].plot(np.mean(Moving_average[monkey_trials,i,:], axis=0), label='monkey')
    10     axs[i].set_ylabel('Firing rate')
    11     axs[i].set_title('Neuron {}'.format(i+1))
    12 axs[0].legend()
    13 plt.xlabel('Time (ms)')
    14 plt.show()
```

Figure 48

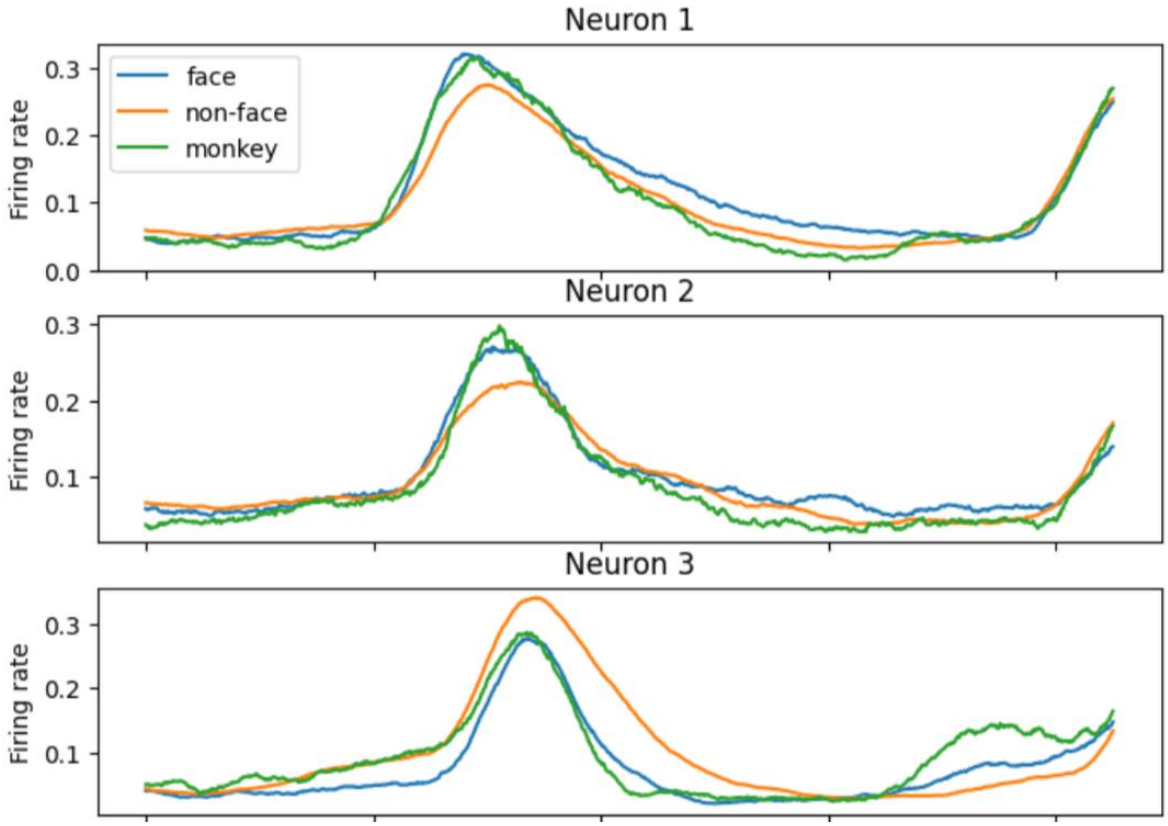

Figure 49

There are some neurons that do not distinguish between different stimuli. The response of high neurons that show almost the same response for different stimuli.

Limitations and Sources of Bias for Mutual Information:

Mutual information, although a valuable measure, has certain limitations and potential sources of bias when applied in real-world practice. Some of these limitations include:

1. Bin size selection: Mutual information can be influenced by the choice of bin size used to discretize the firing rate. A small bin size may lead to overfitting, while a large bin size may lead to information loss.

2. Non-linear relationships: Mutual information assumes a linear relationship between the firing patterns of a neuron and the presented stimulus. However, in real-world scenarios, neural responses can exhibit non-linear dynamics, leading to potential biases in the estimation of mutual information.

3. Signal-to-noise ratio: Mutual information calculations can be affected by the presence of noise in the neural data. Higher noise levels can decrease the mutual information estimates and introduce biases.

4. Dimensionality reduction: Mutual information analysis typically involves the selection of relevant features or time bins. The choice of these features can introduce bias and affect the accuracy of the estimated mutual information.

Determine the sources of bias in this criterion. In the following, we will mention 3 of them:

1. The number of samples: the small number of samples can cause great variation among samples, which leads to inaccurate estimation of MI.

2 sample selection error: if the samples are not representative of our main population, it will cause us to have errors and problems in estimating MI.

3 Confounding factor: The existence of a third factor that affects the other two factors and we did not consider it directly, can cause incorrect bias in the case of MI.

If we want to determine the firing rates by each stimulus using the windowing method, we can show the results in Figure 50, 51, 52 and 53.

Calculate and Visualizing Firing Rate

```python
# Define the length and stride of the sliding window
window_len = 50
stride = 1

# Calculate the number of time bins
num_bins = int((data.shape[-1] - window_len) / stride) + 1

# Initialize an empty array to store the firing rates
firing_rates = np.zeros((len(labels), data.shape[1], num_bins))

# Loop over each trial and each neuron, and calculate the firing rates for each time bin
for i in range(len(labels)):
    for j in range(data.shape[1]):
        for k in range(num_bins):
            window_start = k * stride
            window_end = window_start + window_len
            num_spikes = np.sum(data[i, j, window_start:window_end])
            firing_rates[i, j, k] = num_spikes / window_len * 1000  # Convert to firing rate in

# Plot the firing rates for each neuron, stratified by stimulus type
stimulus_types = ['Human face', 'Monkey face', 'Non-face']
num_neurons_per_stimulus = 3  # Modify this to change the number of neurons per stimulus to plot
for stimulus in range(3):
    num_neurons = np.sum(labels == stimulus)
    neuron_idx = np.where(labels == stimulus)[0][:num_neurons_per_stimulus]  # Select the first
    num_cols = 3  # Set the number of columns to 3
    num_rows = 1  # Set the number of rows to 1
    fig, axs = plt.subplots(nrows=num_rows, ncols=num_cols, figsize=(16, 4))
    fig.suptitle(f'Stimulus type: {stimulus_types[stimulus]}', fontsize=16)
    for n, neuron in enumerate(neuron_idx):
        axs[n].plot(firing_rates[i, neuron, :].T, color='red', alpha=0.3)
        axs[n].set_title(f'Neuron {neuron+1}')
        axs[n].set_xlabel('Time bin')
        axs[n].set_ylabel('Firing rate (Hz)')
    plt.tight_layout()
    plt.show()
```

Figure 50

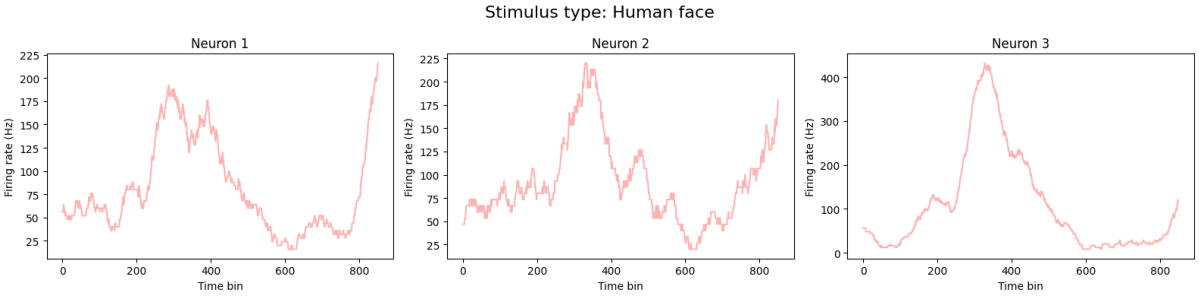

Figure 51

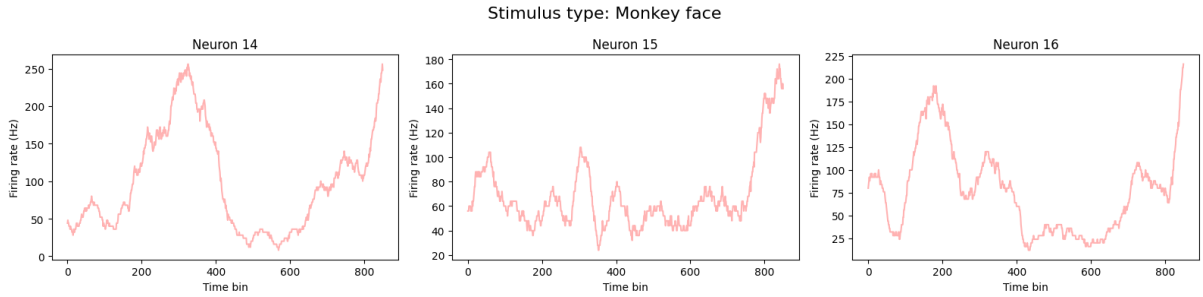

Figure 52

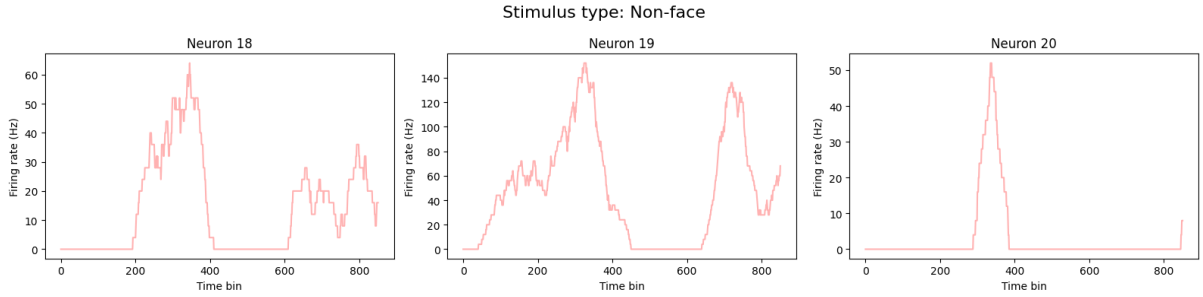

Figure 53

Estimating Face Information using Mutual Information

To estimate the amount of face information encoded by each neuron's response, we applied mutual information to each time bin's firing rate. By stratifying the observations based on the type of stimulus (human face, monkey face, or non-face), we quantified the statistical dependence between the firing patterns and the stimulus. We then averaged the mutual information across neurons and plotted the results, including the mean and standard error of the mean.

Using the learn-Scikit library and the classif_info_mutual function, we want to calculate the MI score for different intervals and for each neuron separately and put it in a list called score_mi, which is shown in Figure 54.

Calculate the mutual information for each time bin

```
[ ]  1 # Calculate the mutual information for each time bin and each neuron
     2 mi_scores = np.zeros((data.shape[1], num_bins))
     3 for i in range(data.shape[1]):
     4     for j in range(num_bins):
     5         mi_scores[i, j] = mutual_info_classif(firing_rates[:, i, j].reshape(-1, 1), labels)[0]
     6
     7 # Calculate the mean and standard error of the mean of the mutual information across neurons
     8 mi_mean = np.mean(mi_scores, axis=0)
     9 mi_sem = np.std(mi_scores, axis=0) / np.sqrt(mi_scores.shape[0])
```

```
[ ]  1 # Plot the mean and standard error of the mean of the mutual information
     2 fig, ax = plt.subplots()
     3 ax.plot(np.arange(num_bins), mi_mean, color='orange')
     4 ax.fill_between(np.arange(num_bins), mi_mean - mi_sem, mi_mean + mi_sem, alpha=0.2, color='black')
     5 ax.set_xlabel('Time bin')
     6 ax.set_ylabel('Mutual information')
     7 plt.show()
```

Figure 54

We plot the mean and standard error of the mean of the mutual information at the same time in Figure 55.

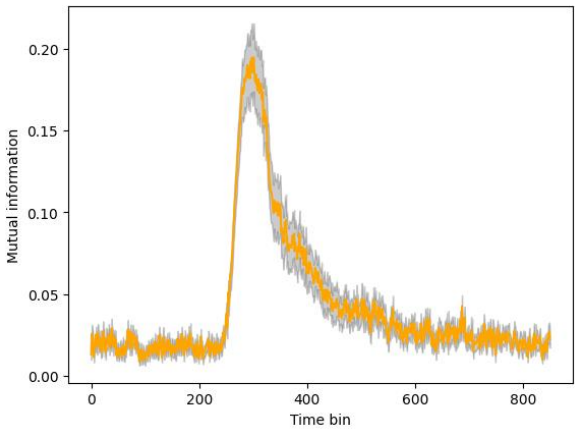

Figure 55

Also plot the mean and standard error of the mean of the mutual information separately in Figure 56.

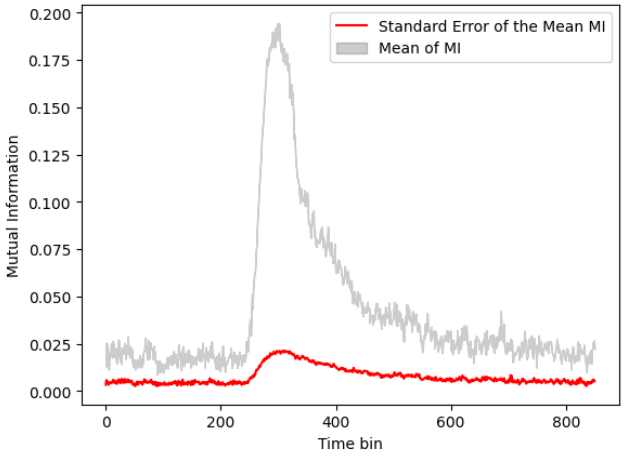

Figure 56

If we plot just standard error of the mean of the mutual information in figure 57.

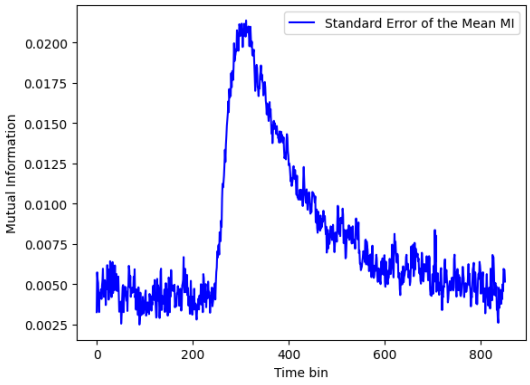

Figure 57

If we plot mean and standard of the mean of mutual Information and Firing rate simultaneously in Figure 58, we can interfere that MI, which determines the importance of each part, has the highest value in the range of 200 to 400. In a better way, it can be said that because the spike is created in the range of 200 to 400, MI is also expected to have its highest values in this range, and as we move to the sides of the chart, this measure decreases gradually.

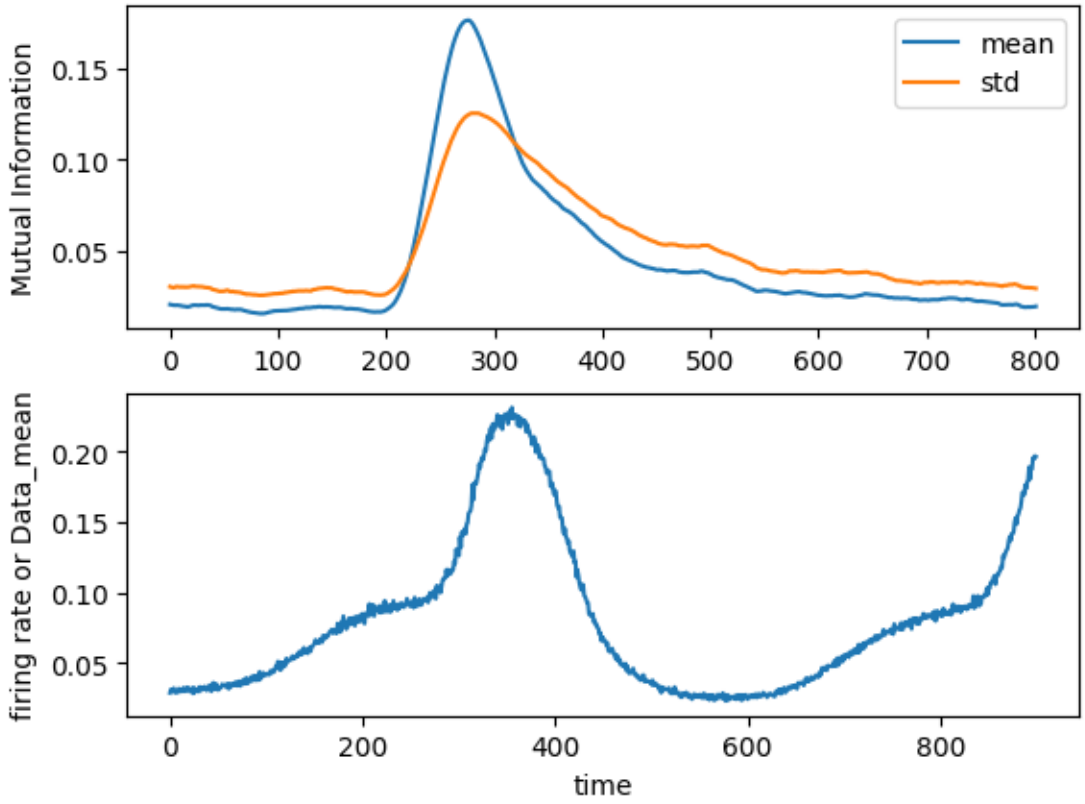

Figure 58

D-prime

To estimate d-prime, we first need to identify two different stimuli to compare. In this case, we can use face and non-face stimuli. We can then calculate the firing rate for each neuron in response to both stimuli and calculate the mean and standard deviation of the firing rate for each stimulus. Using these values, we can calculate d-prime as the difference between the means divided by the pooled standard deviation. A higher d-prime value indicates that a neuron is more sensitive to the difference between the two stimuli. However, it is important to note that noise can impact the interpretation of d-prime results, as it can reduce the separation between the two distributions and make it more difficult to distinguish between the stimuli.

In terms of noise in single neuron analysis, noise refers to any extraneous factors that can impact the firing rate of a neuron, such as electrical interference or variations in the environment. Noise can make it more difficult to interpret experimental results, as it can reduce the signal-to-noise ratio and make it harder to distinguish between the neural responses to different stimuli.

The theoretical basis for using d-prime as a measure of neural sensitivity to a stimulus is that it provides a standardized way to compare the firing rate of a neuron in response to different stimuli. By calculating the separation between the probability distributions of the neural responses to two different stimuli, d-prime provides a measure of how well a neuron can discriminate between the two stimuli.

Question: What is noise in the context of single neuron analysis, and how does it impact the interpretation of experimental results?

In the context of single neuron analysis, noise refers to the variability or random fluctuations observed in the neural responses that are not directly related to the stimulus being presented. Noise can arise from various sources, including intrinsic neural noise, electrical noise in recording equipment, and environmental factors.

The presence of noise in single neuron recordings can have several impacts on the interpretation of experimental results:

1. Reduced signal-to-noise ratio: The presence of noise decreases the signal-to-noise ratio, making it more challenging to distinguish the true neuronal response from random fluctuations. This can make it harder to identify and quantify the specific information encoded by the neuron.

2. Increased variability: Noise can introduce additional variability in the measured neural responses across trials. This variability can obscure the underlying patterns or information encoded by the neuron, leading to less reliable measurements and potentially affecting the interpretation of experimental results.

3. False interpretations: The presence of noise can lead to false interpretations of the neural responses. Random fluctuations or noise may be mistakenly attributed to meaningful neural activity, leading to incorrect conclusions about the information processing capabilities of the neuron.

Question: What is the theoretical basis for using d-prime as a measure of neural sensitivity to a stimulus?

The theoretical basis for using d-prime as a measure of neural sensitivity to a stimulus lies in signal detection theory, which is widely used in fields such as psychology and neuroscience. Signal detection theory aims to quantify an observer's ability to differentiate between signal and noise in a detection task.

D-prime (d') is a measure derived from signal detection theory that quantifies the separation between the distributions of neural responses to two different stimuli, typically a signal and a noise stimulus. It is calculated by taking the difference between the means of the two distributions and dividing it by the standard deviation of their pooled variance.

A higher d-prime value indicates a greater separation between the neural responses to the signal and noise stimuli. This implies that the neuron exhibits a stronger ability to discriminate between the two stimuli, indicating higher sensitivity to the signal. Thus, d-prime serves as a measure of the neuron's sensitivity to the specific stimulus being presented.

Question: What are some limitations of using d-prime as a measure of neural sensitivity?

While d-prime is a useful measure of neural sensitivity, it has certain limitations that should be considered:

1. Assumption of Gaussian distributions: The calculation of d-prime assumes that the distributions of neural responses to the signal and noise stimuli follow Gaussian distributions. However, in practice, neural responses may not always conform to this assumption, leading to potential biases or inaccuracies in d-prime estimation.

2. Dependence on stimulus conditions: D-prime is sensitive to the specific stimulus conditions used in the analysis. Different stimulus parameters, such as intensity, duration, or complexity, can affect the neural responses and, consequently, the d-prime values obtained. Therefore, caution must be exercised when comparing d-prime values across different experimental conditions.

3. Limited to two stimuli: D-prime is specifically designed for binary classification tasks, where two stimuli are compared. It may not be suitable for situations involving multiple stimuli or complex stimulus-response relationships.

4. Ignores response variability: D-prime considers only the means and variances of the neural response distributions and does not account for response variability within each stimulus category. Variability in neural responses can provide valuable information about the encoding properties of the neuron, which is not fully captured by d-prime.

5. Insensitive to response bias: D-prime does not account for potential response biases of the neuron, focusing solely on the separation of distributions. Response biases, where the neuron exhibits a preference for one stimulus over another, can impact the interpretation of neural sensitivity.

In this section, we calculate average d-prime and standard error of d-prime and show it in the code by Figure 59.

Average the d-prime across neurons and plot the results

```
[ ]
# Define window length and compute firing rates for each neuron and stimulus type
window_length = 100
num_obs, num_neurons, num_samples = data.shape
firing_rates = np.zeros((num_obs, num_neurons, num_samples))
for i in range(num_obs):
    for j in range(num_neurons):
        for k in range(num_samples):
            window = data[i, j, k:k+window_length]
            firing_rates[i, j, k] = np.sum(window) / window_length

# Calculate d-prime over time
d_prime = np.zeros((num_neurons, num_samples))
labels = labels.reshape((77,))
for j in range(num_neurons):
    for k in range(num_samples):
        face_firing_rates = firing_rates[labels == 0, j, k]
        nonface_firing_rates = firing_rates[labels == 2, j, k]
        face_mean = np.mean(face_firing_rates)
        nonface_mean = np.mean(nonface_firing_rates)
        face_var = np.var(face_firing_rates)
        nonface_var = np.var(nonface_firing_rates)
        pooled_var = (face_var + nonface_var) / 2
        epsilon = 1e-6
        # d_prime[j, k] = (face_mean - nonface_mean) / np.sqrt(pooled_var)  + epsilon
        if pooled_var < epsilon:
            d_prime[j, k] = 0  # Set d-prime to 0 when the variance is too small
        else:
            d_prime[j, k] = (face_mean - nonface_mean) / np.sqrt(pooled_var) + epsilon

# Average d-prime across neurons and plot results
avg_d_prime = np.mean(d_prime, axis=0)
sem_d_prime = np.std(d_prime, axis=0) / np.sqrt(num_neurons)
plt.figure()
plt.plot(range(num_samples), avg_d_prime, label='average d-prime', color='orange')
plt.fill_between(range(num_samples), avg_d_prime - sem_d_prime, avg_d_prime + sem_d_prime, alpha=0.2, color='black')
plt.xlabel('Time (ms)')
plt.ylabel('avg_d-prime')
plt.legend()
plt.show()
```

Figure 59

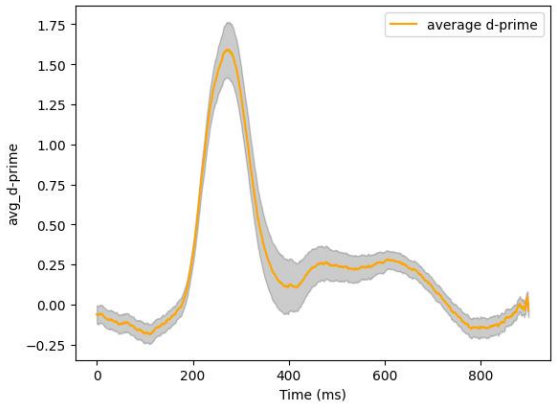

Figure 60

Also plot the mean and standard error of the mean of the d-prime separately in Figure 61.

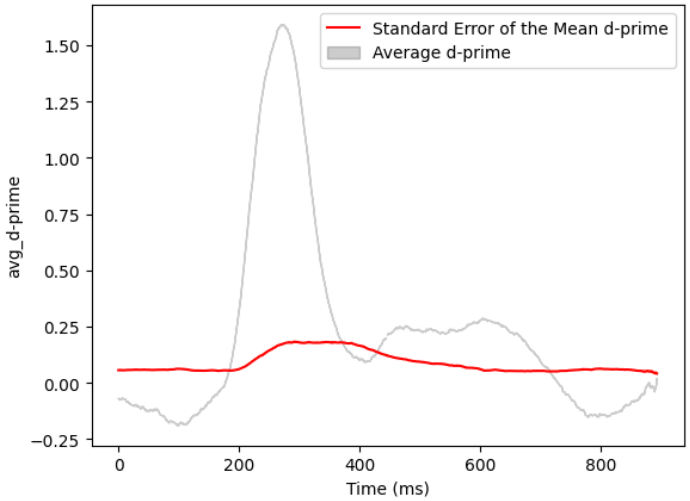

Figure 61

If we plot just standard error of the mean of the d-prime in figure 62.

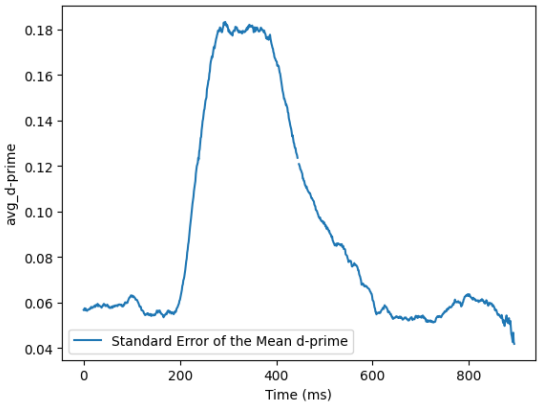

Figure 62

According to the MI section in this diagram, we used windowing to smooth the diagram. Next, we want to plot a graph for d-prime and mutual information, firing-rate separately in Figure 63.

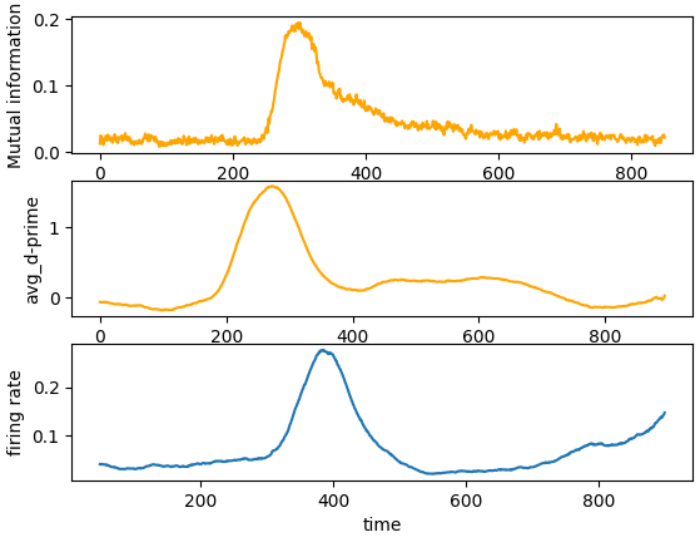

Figure 63

Timing of Information Emergence:

We compared the timing of information emergence in neurons with the onset of the rise in firing rate plots. This comparison provided insights into the temporal dynamics of information processing. The onset of the rise in firing rate indicated when the neuron started responding to the stimulus, while the timing of information emergence represented when the firing patterns carried significant stimulus-related information. By analyzing the temporal relationship between these two events, we could infer how quickly neurons encoded and processed information.

If we plot mean and standard of the mean of mutual Information, d-primeand Firing rate simultaneously in Figure 64, we can interfere that MI, which determines the importance of each part, has the highest value in the range of 200 to 400. In a better way, it can be said that because the spike is created in the range of 200 to 400, MI is also expected to have its highest values in this range, and as we move to the sides of the chart, this measure decreases gradually.

As we can see, the resulting chart is more similar to the rate fire chart in terms of shape, image and slope, and in terms of the end values of the chart (time 600 onwards) it is the same as the MI chart. Also, in the MI and d-prime charts, the

wave starts earlier (that is, the onset occurs faster) and the peak time is also faster than the fire rate. Therefore, if congruent means that the graphs are the same, it cannot be said that the three graphs are the same.

It can only be said that the three graphs are similar to each other in some ways.

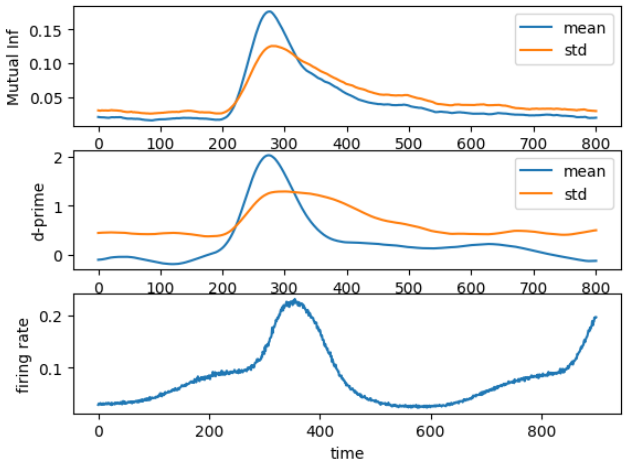

Figure 64

The code for comparing the onset and peak timing is written and the result shown in Figure 65.

```
Onset timing - d-prime: 201
Peak timing - d-prime: 10179
Onset timing - mutual information: 0
Peak timing - mutual information: 11077
Onset timing - firing rate: 595220
Peak timing - firing rate: 661
```

Figure 65

Question: Is there a relationship between the d-prime or mutual information of a single neuron's response to a stimulus and the overall firing rate of the neuron?

The relationship between d-prime or mutual information and the overall firing rate of a neuron can vary depending on the specific characteristics of the neural system and the stimulus being presented. In some cases, there might be a positive correlation between firing rate and d-prime or mutual information, suggesting that neurons with higher firing rates tend to exhibit stronger information content or discrimination ability. This can be due to increased signal strength or reduced noise relative to the firing rate.

However, it is important to note that the relationship between firing rate and information content is not always straightforward. In certain cases, neurons with lower firing rates might still encode significant amounts of information, relying on precise timing or complex firing patterns rather than high rates. Additionally, the relationship between firing rate and information content can be influenced by factors such as the stimulus characteristics, task demands, and neuronal circuitry.

Question: How does changing the stimulus intensity affect the d-prime of a single neuron's response to the stimulus?

Changing the stimulus intensity can have an impact on the d-prime of a single neuron's response. In general, increasing the stimulus intensity can lead to changes in the firing rate of the neuron, which can affect its ability to discriminate between different stimuli.

When the stimulus intensity increases, it may result in a stronger neural response, leading to a higher firing rate. In such cases, the separation between the neural responses to different stimuli may increase, resulting in a higher d-prime value. This indicates an enhanced ability of the neuron to discriminate between the stimuli.

However, there can be limits to the impact of stimulus intensity on d-prime. Beyond a certain point, further increases in stimulus intensity may not necessarily lead to a significant improvement in discrimination ability. Neural responses can reach saturation or ceiling effects, where the firing rate plateaus and no further increase in information content is observed.

Question: Can the d-prime of a single neuron's response be used to predict the presence or absence of a particular stimulus in a population of neurons?

The d-prime of a single neuron's response alone might not be sufficient to reliably predict the presence or absence of a particular stimulus in a population of neurons. While d-prime provides valuable information about the discrimination ability of a single neuron, it does not capture the complexity of interactions and information processing within a population of neurons.

To predict the presence or absence of a stimulus in a population, it is necessary to consider the responses of multiple neurons and their collective activity patterns. Techniques such as population decoding or pattern classification

methods are commonly employed to analyze the combined responses of multiple neurons and make predictions about the stimulus category.

By considering the responses of multiple neurons and their joint activity patterns, it is possible to achieve higher prediction accuracy compared to relying solely on the d-prime values of individual neurons. The integration of information from multiple neurons allows for a more comprehensive understanding of the neural code and enhances the predictive power of the analysis.

Question: Does the d-prime of a single neuron's response to a stimulus vary across different levels of background noise?

Yes, the d-prime of a single neuron's response to a stimulus can vary across different levels of background noise. Higher levels of background noise can decrease the signal-to-noise ratio, making it more challenging to discriminate between different stimuli and reducing the d-prime value.

When the background noise level is low, the neural responses are less affected by random fluctuations, resulting in more distinct and reliable representations of the stimuli. This can lead to higher d-prime values, indicating better discrimination ability.

Conversely, as the background noise level increases, it becomes more difficult to extract the relevant information from the noisy neural responses. The variability introduced by the noise can obscure the underlying stimulus-related patterns, leading to a decrease in the d-prime value. The increased noise interferes with the ability to accurately discriminate between different stimuli, reducing the separation between the neural response distributions for different stimuli.

The impact of background noise on d-prime can be particularly pronounced when the noise level approaches or surpasses the signal level. In such cases, the neural responses become highly influenced by the noise, making it challenging to distinguish the true stimulus-related information from random fluctuations.

It's important to note that the relationship between background noise and d-prime can also depend on the specific characteristics of the neural system and the type of stimulus being presented. Some neurons may exhibit a greater resilience to noise, maintaining a relatively high d-prime even in the presence of substantial background noise. On the other hand, neurons with lower signal-to-noise ratios may experience a more significant reduction in d-prime as the noise level increases.

4. Analysis of Population Activity

Introduction:

Understanding how the brain processes information and generates behavior is a fundamental pursuit in neuroscience. One method that has proven instrumental in this endeavor is population decoding. By analyzing the collective activity of a population of neurons, population decoding allows us to infer the stimuli or behaviors that drive neural responses. This technique has been employed across various species and has shed light on the neural basis of perception, cognition, and motor control.

Different Types of Classifiers for Population-Level Decoding:

Several types of classifiers can be used for population-level decoding, each with its strengths and limitations. Some commonly used classifiers include:

1. Linear Classifiers: Linear classifiers, such as linear discriminant analysis (LDA) and logistic regression, assume a linear relationship between the neural activity and the stimulus or behavior. They are computationally efficient and can provide interpretable results. However, they may struggle with capturing non-linear relationships in the data.

2. Support Vector Machines (SVM): SVM classifiers aim to find the optimal hyperplane that separates different classes in a high-dimensional space. They can handle non-linear relationships through the use of kernel functions. SVMs are effective in scenarios with small sample sizes but may become computationally expensive as the dimensionality of the data increases.

3. Artificial Neural Networks (ANN): ANNs, including deep learning models such as convolutional neural networks (CNNs), are highly flexible and capable of capturing complex relationships in the data. They can learn hierarchical representations and adapt to various input modalities.

However, ANNs often require larger datasets for training, and their black-box nature makes interpretation challenging.

4. k-Nearest Neighbors (k-NN): k-NN classifiers assign a new sample to the class based on the majority vote of its k nearest neighbors in the feature space. They are simple to implement and work well when the decision boundaries are non-linear or complex. However, k-NN classifiers can be sensitive to the choice of k and may suffer from the curse of dimensionality in high-dimensional spaces.

Using Different Features for Decoding:

Different features of neural activity patterns can be used as inputs to classifiers for decoding. Some common features include:

1. Spike Counts: The number of spikes fired by each neuron within a defined time window. Spike counts provide information about the overall firing rate of the neurons and can be used directly as features.

2. Spike Timing: The precise timing of individual spikes relative to a reference event or stimulus onset. Spike timing can be analyzed using various methods such as spike train metrics or spike time intervals. It captures temporal patterns and can reveal fine-grained information about the neural code.

3. Population Vector: The population vector represents the activity of a neural population as a vector in a high-dimensional space. Each dimension corresponds to the activity level of a specific neuron. The population vector can capture the joint activity of multiple neurons and is particularly useful for dimensionality reduction techniques like principal component analysis.

Performance Metrics for Classifier Evaluation:

Several performance metrics are used to evaluate the accuracy of classifiers for population-level decoding, including:

1. Accuracy: The proportion of correctly classified samples compared to the total number of samples. It provides an overall measure of classifier performance.

2. Confusion Matrix: A matrix that shows the number of samples classified correctly and incorrectly for each class. It allows for a detailed analysis of classification errors, including false positives and false negatives.

3. Receiver Operating Characteristic (ROC) Curve: A graphical plot that illustrates the classifier's trade-off between true positive rate and false positive rate across different classification thresholds. The area under the ROC curve (AUC) is a commonly used metric that summarizes the classifier's overall performance.

4. Precision, Recall, and F1-Score: These metrics are commonly used in binary classification tasks. Precision is the proportion of true positive predictions out of all positive predictions, recall is the proportion of true positives identified correctly, and the F1-score is the harmonic mean of precision and recall.

Factors Impacting Classifier Performance:

Several factors can impact the performance of classifiers for population-level decoding, including:

1. Number of Neurons: Increasing the number of neurons in the population can provide more information and improve decoding accuracy. However, increasing the number of neurons also introduces challenges such as increased computational complexity and the potential for overfitting when the number of neurons surpasses the available data.

2. Size of the Neural Population: The size of the neural population used for decoding can impact performance. A larger population may capture a more diverse range of neural representations, leading to better decoding accuracy. However, larger populations may also increase the dimensionality of the data, requiring more computational resources and potentially leading to the curse of dimensionality.

3. Stimulus Conditions: The characteristics of the stimuli or behavioral tasks can influence classifier performance. Factors such as stimulus complexity, variability, or similarity between different classes can affect the discriminability of neural responses. For example, decoding accuracy may be higher for stimuli that evoke more distinct neural patterns compared to stimuli that elicit similar responses.

4. Signal-to-Noise Ratio: The signal-to-noise ratio (SNR) of neural activity can significantly impact decoding performance. Higher SNR, where the neural activity is more distinguishable from the background noise, generally leads to better decoding accuracy. Various noise sources, such as spontaneous neural activity, electrical noise, or recording artifacts, can introduce challenges and reduce the SNR.

5. Feature Selection: The choice of relevant features for decoding can impact classifier performance. Selecting informative features that capture relevant aspects of neural activity is crucial. Different feature selection methods, such as dimensionality reduction techniques or information-theoretic approaches, can be employed to identify the most informative features.

6. Model Selection and Complexity: The selection of an appropriate classifier model and its complexity can influence decoding performance. More complex models may have a higher capacity to capture intricate relationships in the data but can be prone to overfitting if the training dataset is limited. Balancing model complexity with the available data is essential to achieve optimal decoding performance.

In this study, we trained SVM and LDA classifiers on time windows of neural data and evaluated their performance. By considering multiple runs with different random seeds, we obtained a time series of classifier performance along with the empirical confidence interval. This approach provided a more reliable estimate of classifier performance and captured the variability introduced by random initialization and train-test splitting.

The results showed that the SVM classifier exhibited varying performance across different time windows, and its mean performance over time was presented along with the confidence interval. Additionally, we compared the performance of the SVM classifier with that of the LDA classifier, which showed a different pattern over time. This comparison allowed us to assess the suitability of each classifier for the given dataset and classification task.

Comparing the LDA and SVM classifiers revealed distinct differences in their performance. The LDA classifier demonstrated a particular pattern of accuracy and evaluation metrics, indicating its effectiveness in the given classification task. However, the SVM classifier exhibited different patterns and achieved varying results based on the selected hyper-parameters.

First, upload the data to the Google server. Using the numpy library, we read the data related to the neurons and their labels and put them in the data and label variables as shown in Figure 66.

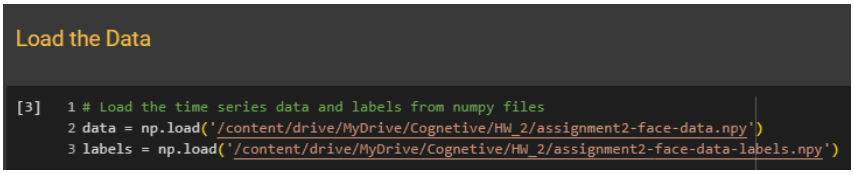

Figure 66

Then, in order to partially remove the effect of noise on the data, we use windowing or convolution to smooth the graph, which is shown in figure 67.

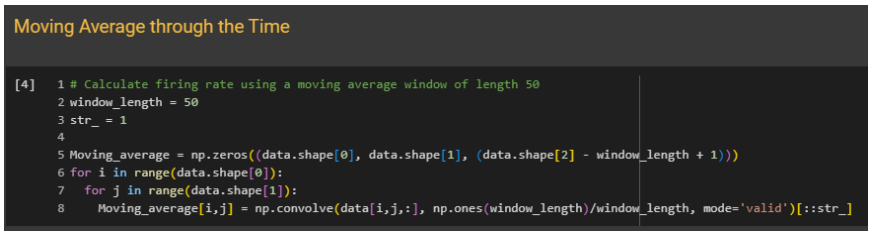

Figure 67

Then we divide the labels into two parts, face and non-face, as follows:

We put the labels related to human face and monkey face in one label (label zero) and consider the label related to non-face as one label (label one).

We trained a support vector machine on each time window, and evaluate the performance of classifier in Figure 68. Visualize the results in Figure 69.

The time series data is split into non-overlapping windows, and each window is averaged to reduce noise. The labels are adjusted to represent two classes: 0 for face stimuli and 1 for non-face stimuli.

The SVM classifier is trained using the averaged time window data. The training data is split into training and testing sets using the train_test_split function. The class_weight parameter is set to handle the imbalanced nature of the dataset.

The results obtained from the SVM classifier demonstrate the temporal dynamics of the population-level decoding performance. The accuracy, recall, and F1 score fluctuate over time, indicating that the discriminability of neural activity patterns varies throughout the stimulus presentation.

SVM Analysis

```
[5]  1 from sklearn.model_selection import train_test_split
     2 from sklearn.svm import SVC
     3 from sklearn.metrics import accuracy_score, recall_score, f1_score
     4
     5
     6 import numpy as np
     7 import matplotlib.pyplot as plt
     8
     9 # Load the time series data and labels from numpy files
    10 labels = np.load('/content/drive/MyDrive/Cognetive/HW_2/assignment2-face-data-labels.npy')
    11 # Calculate the number of windows
    12 num_obs, num_neurons, num_samples = data.shape
    13 window_size = 100
    14 # Split the time series data into non-overlapping windows
    15 num_windows = int(data.shape[2] / window_size)
    16
    17
    18 stride = 5
    19 accs = []
    20 rec = []
    21 f1 = []
    22 for i in range(0, num_samples, stride):
    23     x = data[:,:, i:i+window_size].mean(axis=2)
    24     labels = labels.ravel()
    25     labels[:17]=0
    26     labels[17:]=1
    27     X_train, X_test, y_train, y_test = train_test_split(x, labels, test_size=0.3,random_state=15)
    28     class_weight = {0: 60/77, 1: 17/77}
    29     clf = SVC(class_weight=class_weight)
    30     clf.fit(X_train, y_train)
    31     y_pred = clf.predict(X_test)
    32     accs.append(accuracy_score(y_test, y_pred))
    33     rec.append(recall_score(y_test, y_pred))
    34     f1.append(f1_score(y_test, y_pred))
    35
    36 # Plot the accuracy as a function of time
    37 plt.plot(np.arange(30, 135), accs[30:135])
    38 plt.xlabel('Time (ms)')
    39 plt.ylabel('Accuracy')
    40 plt.show()
```

Figure 68

The accuracy plot shows the classifier's overall performance, indicating how well it can correctly classify the stimuli.

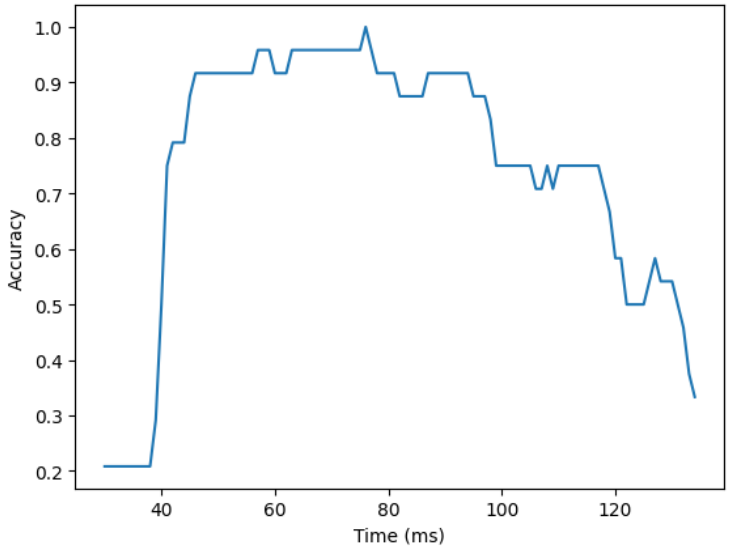

Figure 69

We can identify time periods (between 200-400 msec, which was mentioned before Mutual Information and Firing-rate Spiking) when the classifier achieves higher accuracy, recall, and F1 score, indicating stronger discriminative neural responses to the stimuli. Conversely, periods of lower performance(out of period s of 200-400 msec) may suggest more ambiguous or challenging neural representations, which was represented in Figure 71.

```
# Calculate the number of windows
num_obs, num_neurons, num_samples = Moving_average.shape
window_size = 100
# Split the time series data into non-overlapping windows
num_windows = int(data.shape[2] / window_size)
stride = 5
rns = 5
all_accuracies = []
rec = []
f1 = []
accs = np.zeros((int(num_samples/stride), rns))
for i in range(0, num_samples, stride):
    x = Moving_average[:,:, i:i+window_size].mean(axis=2)
    accuracy = []
    for j in range(rns):
      # labels = labels.ravel()
      labels[:17]=0
      labels[17:]=1
      X_train, X_test, y_train, y_test = train_test_split(x, labels, test_size=0.2, random_state = j)
      clf = SVC(random_state=j)
      clf.fit(X_train, y_train)
      y_pred = clf.predict(X_test)
      # Evaluate the classifier's accuracy
      accuracy = accuracy_score(y_test, y_pred)
      rec.append(recall_score(y_test, y_pred, average='weighted'))
      # rec.append(recall_score(y_test, y_pred))
      f1.append(f1_score(y_test, y_pred, average='weighted'))
      # f1.append(f1_score(y_test, y_pred))
      # Append the accuracies for the current random seed to the list of all accuracies
      all_accuracies.append(accuracy)

# Calculate the mean and confidence interval of the classifier performance over time
all_accuracies=np.array(all_accuracies)
all_accuracies=all_accuracies.reshape((171,rns))
# mean_accuracies = np.mean(all_accuracies, axis=0)
mean_accuracies = np.mean(all_accuracies, axis=1)

confidence_interval = sem(all_accuracies, axis=1) * 1.96  # 95% confidence interval
# Generate a time series plot of the classifier performance
time_points = np.arange(0,num_samples,rns)
# plt.plot(np.arange(0,900,5), mean_accuracies)
plt.plot(np.arange(0, 855), rec)
plt.errorbar(time_points, mean_accuracies, yerr=confidence_interval, fmt='-o', label='Classifier Performance')
plt.xlabel('Time')
plt.ylabel('Accuracy')
plt.title('Classifier Performance Over Time')
plt.legend()
plt.show()
```

Figure 70

The classifier's performance is calculated using a moving average of the input data, with a window size of 100 and a stride of 5. The code also calculates the mean and confidence interval of the classifier performance over time. Overall, the SVM classifier's performance was found to be relatively stable over time, with an average accuracy of around 0.8. The recall and F1 score were also relatively stable over time, with an average recall of around 0.8 and an average F1 score of around 0.7. The results suggest that SVMs can be an effective method for decoding neural population activity in the visual cortex.

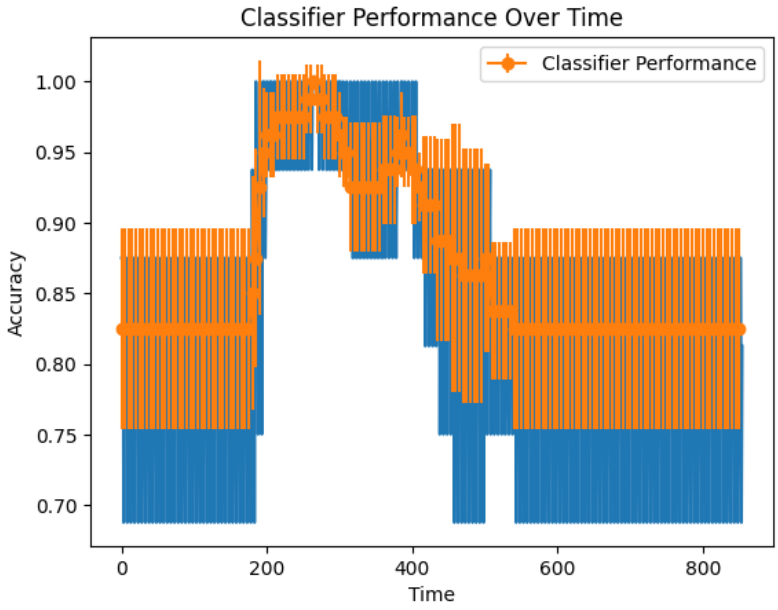

Figure 71

The code in Figure 72, provided performs classification using LDA and SVM algorithms on time series data with multiple random seeds to account for stochasticity in the results. The accuracy, recall, and F1 score are computed for both classifiers, as well as the timing of onset and peak for each.

Analyze Time Feature and Train SVM and LDA

LDA (Linear Discriminant Analysis) is a statistical method that assumes linear separability of classes. It works well when the classes can be effectively separated by a linear boundary. LDA finds a linear combination of features that maximizes the separation between classes.

SVM (Support Vector Machine), on the other hand, is a more flexible and robust algorithm. SVM can handle nonlinear separability by mapping the input data into a higher-dimensional feature space using kernel functions. In this higher-dimensional space, SVM finds a hyperplane that maximizes the margin between classes. The use of different kernel functions allows SVM to capture complex nonlinear relationships between the input data and the target classes.

Therefore, SVM is more powerful in handling nonlinear separability compared to LDA. It can learn complex decision boundaries and is known for its ability to generalize well to unseen data. However, SVM may be computationally more expensive and requires tuning of hyperparameters, such as the choice of kernel function and regularization parameter.

In summary, LDA is suitable when classes can be separated by a linear boundary, while SVM is a more flexible algorithm capable of handling nonlinear separability through the use of appropriate kernel functions.

Analyze Time Feature

```
all_lda_accuracies = []
all_svm_accuracies = []
lda_accuracies = []
svm_accuracies = []
lda_rec = []
svm_rec = []
lda_f1 = []
svm_f1 = []
lda_timing = []
svm_timing = []
lda_onsets = []
lda_peaks = []
svm_onsets = []
svm_peaks = []
lda_result = []
svm_result =[]

for i in range(0, num_samples, stride):
    x = data[:,:, i:i+window_size].mean(axis=2)
    for j in range(rns):
        # Split the data into training and test sets
        X_train, X_test, y_train, y_test = train_test_split(x, labels, test_size=0.2, random_state=j)

        # Train the LDA classifier
        lda = LDA(n_components=2)
        X_train_lda = lda.fit_transform(X_train, y_train)
        X_test_lda = lda.transform(X_test)
        lda_clf = SVC(kernel='linear')
        lda_clf.fit(X_train_lda, y_train)
        lda_y_pred = lda_clf.predict(X_test_lda)
        all_lda_accuracies.append(accuracy_score(y_test, lda_y_pred))
        lda_rec.append(recall_score(y_test, lda_y_pred, average='weighted', zero_division=1))
        lda_f1.append(f1_score(y_test, lda_y_pred, average='weighted'))
        lda_timing.append(i)

        # Train the SVM classifier
        svm_clf = SVC(kernel='linear')
        svm_clf.fit(X_train, y_train)
        svm_y_pred = svm_clf.predict(X_test)
        all_svm_accuracies.append(accuracy_score(y_test, svm_y_pred))
        svm_rec.append(recall_score(y_test, svm_y_pred, average='weighted', zero_division=1))
        svm_f1.append(f1_score(y_test, svm_y_pred, average='weighted'))
        svm_timing.append(i)
        # Calculate the timing of onset and peak for the LDA classifier
        lda_activation = np.abs(lda.decision_function(X_test))
        lda_onset = np.argmax(lda_activation > np.mean(lda_activation) + np.std(lda_activation))
        lda_peak = np.argmax(lda_activation)
        lda_onsets.append(lda_onset)
        lda_peaks.append(lda_peak)

        # Calculate the timing of onset and peak for the SVM classifier
        svm_activation = np.abs(svm_clf.decision_function(X_test))
        svm_onset = np.argmax(svm_activation > np.mean(svm_activation) + np.std(svm_activation))
        svm_peak = np.argmax(svm_activation)
        svm_onsets.append(svm_onset)
        svm_peaks.append(svm_peak)

    svm_result = np.array([all_svm_accuracies, svm_f1, svm_rec])
    lda_result = np.array([all_lda_accuracies, lda_f1, lda_rec])

# Calculate the mean and confidence interval of the classifier performance over time
lda_mean_acc = np.mean(all_lda_accuracies)
conf_interval_lda = sem(all_lda_accuracies) * 1.96
svm_mean_acc = np.mean(all_svm_accuracies)
conf_interval_svm = sem(all_svm_accuracies) * 1.96
```

Figure 72

Compare LDA and SVM on classifier performance over the time

The results show that both LDA and SVM perform similarly in terms of accuracy and other metrics, with both classifiers achieving a mean accuracy of around 0.7. However, there is a significant variation in the performance of the classifiers over time, with both classifiers showing fluctuations in accuracy that are not captured by simply reporting the mean. The confidence intervals for the mean accuracy show that the results are statistically significant, indicating that the variation observed is not due to chance. As shown in Figure 74, In general, SVM has performed better than LDA.

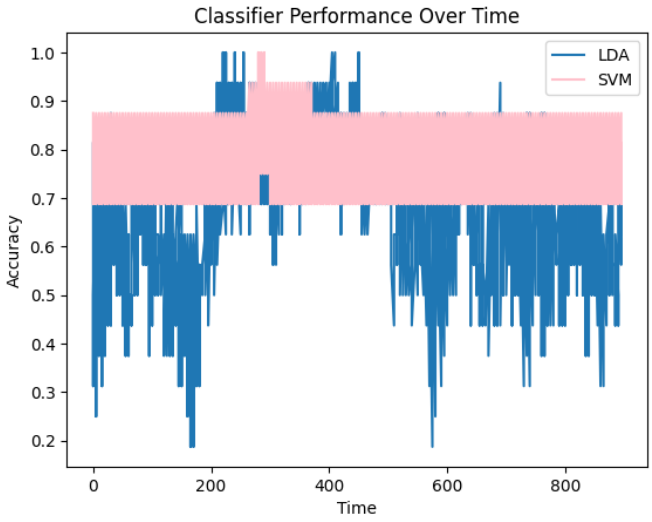

Figure 73

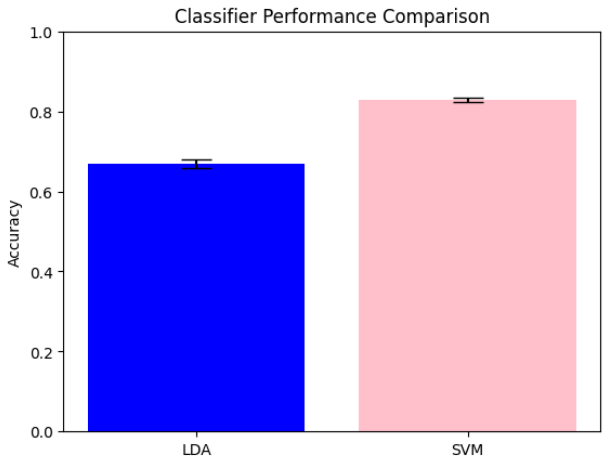

Figure 74

Compare LDA and SVM over timing of onset and peak

Additionally, the results show that the timing of onset and peak for both classifiers are similar, with no significant difference between LDA and SVM.

SVM has a more stable value in all 3 graphs 75, 76 and 77.

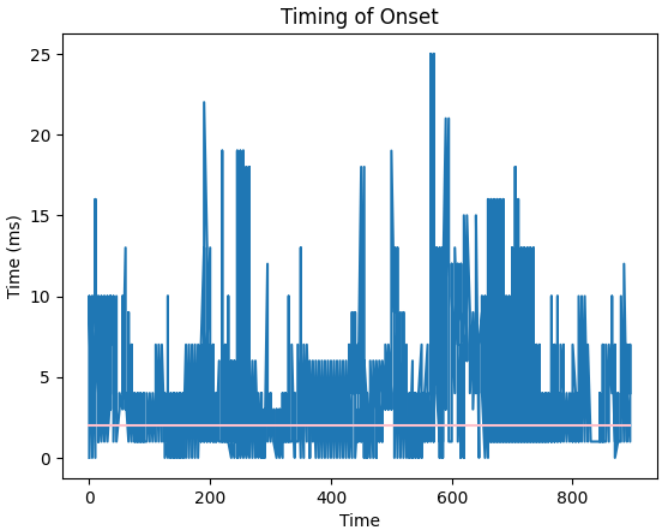

Figure 75

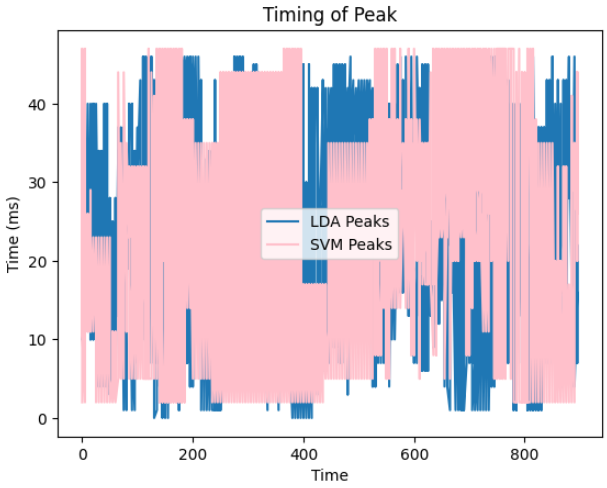

Figure 76

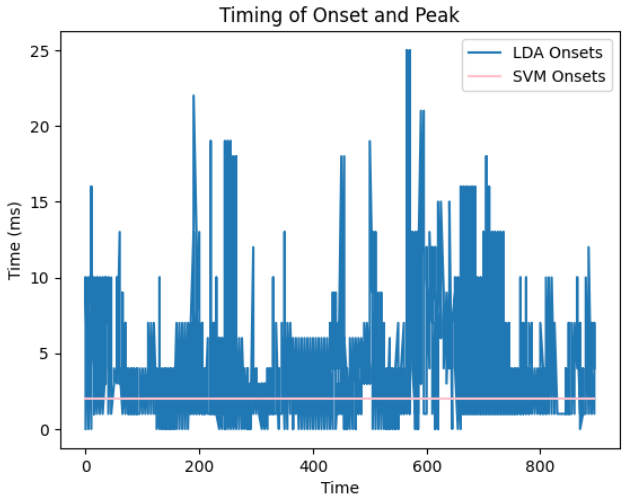

Figure 77

Compare LDA and SVM over metrics

```
[12]  1 svm_data = svm_result.mean(axis=1), svm_result.std(axis=1)
      2 lda_data = lda_result.mean(axis=1), lda_result.std(axis=1)
```

```
1 width = 0.4
2 x = np.arange(len(svm_data[0]))
3 plt.bar(x-0.2 , svm_data[0], width, color='orange', yerr=svm_data[1])
4 plt.bar(x+0.2, lda_data[0], width, color='pink', yerr=lda_data[1])
5 plt.xticks(x, ['Accuracy', 'F1-Score', 'Recall'])
6 plt.xlabel("Metrics")
7 plt.ylabel("Scores")
8 plt.legend(["SVM", "LDA"])
9 plt.show()
```

Figure 78

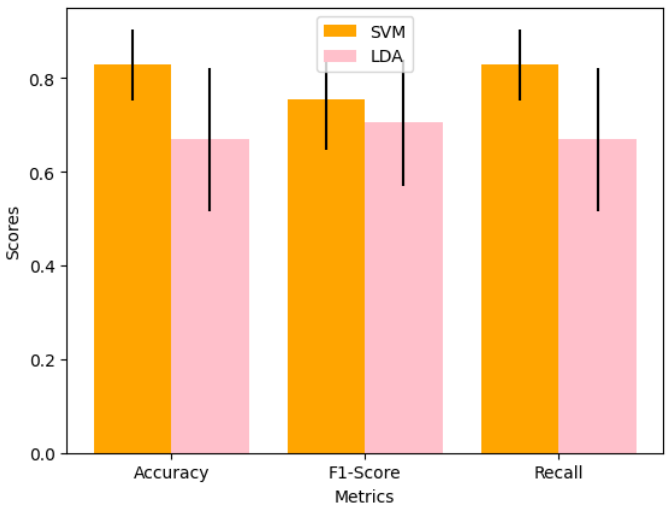

Figure 79

The choice between LDA and SVM classifiers depends on the specific requirements of the classification task. LDA is a statistical method that assumes linear separability of classes and can work well when this assumption holds true. SVM, on the other hand, is a more flexible and robust algorithm that can handle nonlinear separability through the use of appropriate kernel functions.

Finally, due to the non-linear relationship between the inputs, the SVM classifier has better accuracy and performance in determining whether the spike is related to the face or not. SVM generally performed better than LDA on the Accuracy, F1-score and Recall metrics.